Tools, labs, and level conversion

Tools, labs, and level conversion

This part contains 33 notes, from `tools/build-diff/README.md` through `live-runs/2gta-222-2026-10-30/README.md`.

Notes under `tools/` and `live-runs/`, in path order. `tools/navigator/README.md` describes the notes viewer. The viewer program stays in the source tree.

`tools/build-diff/README.md`

build-diff

Cross-build function matcher used for `client-internals/build-diff-204-205.md`.

1. In Ghidra (headless), after analysis and name application, run `ExportFeatures.java <out.jsonl>` on each build.
2. Run `python3 fdiff.py old.jsonl old.exe new.jsonl new.exe out.json [Class1,Class2,...]`. The optional class list turns off virtual-slot name matching for classes whose VMT shifted between the builds.

The output lists identical, changed (with instruction-stream similarity and string-literal deltas), only-old and only-new functions.

`tools/classic-tools/character-designer.md`

Graal Character Designer and GChar

Date: 2026-11-01

GCD 1.2.7 (November 1999, Andrew Bridges / Bruges) and GChar 1.0 release CA, the same author. Both are Delphi windows for classic character looks. Neither image imports a socket library. **[bin][doc]**

`chrfile.py` parses the character line.

Files they read

Both expect a Graal folder that contains `headsconfig.txt`, `levels\heads\`, `levels\gifs\` and `levels\webgifs\`. If `headsconfig.txt` is missing they say so and stop. GCD also looks for `Graal.exe` under the usual install folders and, failing that, asks the operator to type the folder. **[bin]**

GChar reads `GChar.ini` from its own folder: `drawmethod` (1 or 2), `headmode` (1 or 2) and `graaldir` (the folder the operator set). The command `/setdraw 1` or `/setdraw 2` writes the draw method and shows `Draw Mode Set to 1` or the matching line for 2. Draw mode 2 is the default. Head mode 1 is a head number. Head mode 2 is a gif name with no `.gif`, looked up under `levels\heads` and `levels\webgifs`, or a file the operator picks. The readme says a typed name is not checked beyond being a gif, and that a head with no number is not one the server has. **[doc][file][bin]**

On startup each tool looks for `*.chi` in the Graal folder, adds characters that are not already in the list, and renames those files to `*.chr` so the next run does not add them again. GChar warns and skips a CHI whose name is already present; the operator has to open it and use Save to Graal. **[doc]**

`GCD.dat` and `GChar.dat` are small binary scratch files (the GChar readme mentions a fix for that file being corrupted). They are not the character format. **[file]**

The character line

A `.chr` or `.chi` file is one text line. Every sample in both archives has this shape: **[file]**

```
CHR:1.00Blade=head189.gif,male,black,black,gray,gray,red
```

`CHR:1.00`, the name, `=`, the head gif, sex (`male` or `female`), and five colour names. The file does not label the five colours. Sample names include black, gray, red, yellow, darkblue, cynober, brown, green, purple, lightblue, darkgreen, orange, darkred, darkpurple, white, lightgray, pink and lightgreen. Two samples are `female` (Anna, Sarah). Several files whose names say Girl are stored as `male`; the head gif is doing the work. **[file]**

What the operator sees, and what is never sent

Buttons along the bottom set the colours. A box on the right is the head. Buttons on the left turn the preview. Name sets the name. Save to Graal writes the character into the client's list. Save to file writes `.chr` or `.chi`. Open reads a file. Load reads one back from the Graal list. Snapshot writes a bmp of the current view. Manage deletes, reorders and renames list entries. **[doc]**

Nothing is sent. There is no list login, no account, and no URL request in the import table. The readme's contact line is a mailbox on the official site; the programs do not open it. **[bin][doc]**

`tools/classic-tools/graal-level-gen.md`

GraalLevelGen (December 2000)

Date: 2026-11-01

The Delphi generator shipped as `GraalLevelGen.exe` with `levelgendoc.html` and `levelgen.png`. The one-line note in `../generators.md` calls it an image-to-levels writer. This is the operator contract from that HTML, checked against the image. The image imports no socket library. **[doc][bin]**

The package readme says the levels are New World format and that the tool does not apply to 1.3.9 or older. **[doc]**

What the operator does

Draw a map whose width is 64 times the number of levels across and whose height is 64 times the number of levels down. Save it as a gif, or as an 8-bit png. Start the tool, pick the image, an output folder and a world name, and press Generate World. **[doc]**

Colour	RGB	Becomes
Green	24, 140, 24	Grass, not drawn
Light green	181, 239, 189	Forest
Blue	0, 0, 255	Water

Colour	RGB	Becomes
Maroon	128, 0, 0	Mountains
Red	255, 0, 0	Flowers
Light brown	150, 110, 0	Sand
Brown	113, 82, 0	Small sand stones
Dark gold	83, 61, 0	Big sand stone
Dark green	0, 106, 0	Swamp
Very dark green	0, 66, 0	Bushes
Cyan	0, 255, 255	Puddle
Dark cyan	0, 179, 179	Puddle stone

The mapping from a pixel to a tile is a grid, not one tile per pixel, so the levels are not a 1:1 copy of the picture. **[doc]**

What it writes

Files named `worldname_a-01.graal` through, at the largest, `worldname_z-99.graal`, plus `bigmap.txt` for the editor's map (the M key). The HTML says not to drop that `bigmap.txt` on top of the main world's copy. **[doc]**

A second run on levels that already exist reads them and adds the new objects. It does not replace the previous board. That is how the doc stacks trees on mountains: generate the mountains, generate again for the trees. **[doc]**

The image contains the signatures `GLEVNW01` and `GR-V1.00` through `GR-V1.03`, and the words `BOARD`, `LINK`, `SIGN` and `NPC`. Those strings sit in the same editor unit as level names such as `start1.graal`. Which signature a given save writes was not separated from that unit. The HTML is clear about the file names. **[bin][doc]**

What it never sends

No socket import, no login, no server list. The world stays in the output folder. **[bin]**

`tools/classic-tools/gserverstat.md`

gserverstat and gserverstat2

Date: 2026-11-01

Two ELFs from the 1999 gserver tree, plus the `statmaker` shell script that runs them. They print an HTML leaderboard on standard output. Neither image imports a socket call. **[bin]**

`account13.py` reads and writes the account blob and builds a page row. The row has no password.

What the operator runs

`statmaker` loops forever. Each step runs one ELF with one mode name and moves the HTML into a `stats/` tree (admin modes under `stats/admin/`). It sleeps 10 seconds between pages and 600 seconds at the end of the loop. **[file]**

Program	Modes the script asks for	Page
`gserverstat`	kills, deaths, rupees, highap, lowap, onlinetime	`stats/<mode>.html`

Program	Modes the script asks for	Page
`gserverstat2`	swordpower, maxpower, glovepower, bombpower, shieldpower, highap	`stats/admin/<mode>.html`

Both images contain the same 14 mode names, matched without regard to case. With no argument the mode is kills. **[bin]**

Index	Name	Field, highest first unless noted
0	Maxpower	max hearts
1	Power	current hearts
2	Rupees	rupees
3	Arrows	arrows
4	Bombs	bombs
5	Glovepower	glove
6	Bombpower	bomb power
7	Swordpower	sword
8	Shieldpower	shield
9	Kills	kills
10	Deaths	deaths
11	Onlinetime	online seconds
12	Highap	alignment, highest first; a tie keeps fewer kills first
13	Lowap	alignment, lowest first; a tie keeps more kills first

What it reads

The working directory is the server directory. The loader shared with that era's gserver reads `weapons.txt`, `accountslist.txt`, and, if present, `accounts.txt` (that file is removed after import). Each account file is `accounts/acc` plus the file number from the list. **[src][bin]**

`accountslist.txt` is one account per line: length-prefixed name, length-prefixed password, a flag byte, and a three-byte file number. The flag byte is bit 0 only-load, bits 1–3 admin level, bit 4 blocked. **[src]**

`accounts/accN.txt` is the blob `account13.py` parses: name, password, property list, the same flag byte, two unused zero counts, weapons, action flags, opened chests. The property list is a sequence of `id + 32` and a value. The leaderboard ids used here are nickname (0), max hearts (1), rupees (3, three 7-bit bytes), kills (27), deaths (28), online seconds (29) and alignment (32). **[src]**

What it shows

A green page (`#009933`, background `image/fond2b.gif`) and a table 410 pixels wide. The header cells are Place, Name, and the mode title. Each data row is the place in red, the nickname, and the value in black. An empty nickname is left blank. At most 50 rows. **[bin]**

Skipped before ranking: an account with the only-load flag set, and the account whose name is exactly `super`. **[bin]**

What it never sends

No socket, so no login and no list request. The HTML does not include the password or the account file number. The script only redirects standard output into those HTML files. **[bin][file]**

`tools/classic-tools/README.md`

Classic server-side tools

Date: 2026-11-01

Plain behaviour of the tools whose earlier summaries stopped at a purpose line. Static reads only: strings, import tables, and named functions. No image was executed. Tags: **[bin]** the image, **[src]** the 1999 account writer that the stat tools link, **[file]** a sample file in the tool's own archive, **[doc]** that archive's readme.

Note	Tool	What was new
`serverlist3.md`	Classic listserver ELF	Config keys, the zlib frame, what a client sends, what a game server sends, what is never sent.
`gserverstat.md`	`gserverstat` and `gserverstat2`	The 14 page modes, the top-50 rules, the account file, the HTML columns.
`character-designer.md`	GCD 1.2.7 and GChar 1.0	The `CHR:1.00` line, CHI versus CHR, and that neither tool opens a socket.
`graal-level-gen.md`	GraalLevelGen (December 2000)	Image rules, colour table, output names, merge-on-second-run.
`text-convert.md`	TextConvert	Folder walk, GR-V1 in, GLEVNW01 out, no socket.

In-process references, with tests in `test_classic_tools.py`:

Module	Recovers
`listframe.py`	The listserver frame and the client and game-server login lines.
`account13.py`	The `accounts/accN.txt` blob and a leaderboard row with no password.
`chrfile.py`	A `CHR:1.00` character line.

Skipped

- Win32 `ListServer.exe`. Already described in `../server-tools-2001.md`, including the saved-login hand-off. Not reimplemented.
- Trainers. Another branch owns those notes.
- `GServer Stats.zip` on Drive. Blocked as an abusive download. The stat notes come from the small gserver source archive's `gserverstat`, `gserverstat2` and `statmaker` instead.
- GraalLevelGenNW. The December 2000 generator's own HTML now covers the operator contract, and the NW summary already says that build is offline and writes a `levels` tree.
- Remote control, NPC control, Guildmaker, the later C# listserver. Existing notes already answer what they send.

`tools/classic-tools/serverlist3.md`

Classic listserver (`serverlist3`)

Date: 2026-11-01

The ELF in the Server Lister package. One process, two listen ports, MySQL for accounts and the public directory. It is not the Win32 `ListServer.exe` in `../server-tools-2001.md`. **[bin]**

`tools/classic-tools/listframe.py` packs and unpacks the frame and the login lines. It does not open a socket.

Start and config

The only argument is the full path of a config file. If it is missing, the process prints `Configuraton not found.` and `Pleace specify the full path of the configuration file as parameter.` and exits. Those spellings are in the image. **[bin]**

Each config line is `key=value`. The keys it reads are:

Key	Role
`clientsport`	Port for game clients
`serversport`	Port for game servers
`playerlimit`	Cap used by the free-player check
`databaseserver`	MySQL server name
`showmore`	URL sent to a client on the trial and free-server replies

A sample file in the same package sets the two ports to the lab client ports 14922 and 14923 and a player limit. The show-more value is whatever URL the operator wrote. **[file]**

On the console it prints `Server lister started` with the two ports. Beside the process it writes `playercount.txt`, `maxplayers.txt`, `accountcount.txt` and `playerstats.txt`. The stats file has `Servers:` and `Players:` sections, marks a player `upgraded`, `ghost` or `trial`, and a summary line for waiting ghosts, ghosts, free players and the limit. **[bin]**

Frame

Both listeners use one framing. A frame is a big-endian 16-bit length and then that many zlib bytes. The inflated body is newline-separated lines. The first byte of a line is `id + 32`. A length-prefixed string is `count + 32` and then that many bytes. An uncompressed buffer past 16383 bytes, or an outgoing buffer past 4095 bytes, is treated as a broken link (the outgoing buffer is flushed first). **[bin]**

What a client sends

Id	Rest of the line
0	Edition text. Stored and used to filter the directory.
1	Account, then password, each a length-prefixed string.
other	Ignored.

There is no gen-5 key, no version token of the later list, and no second cipher. **[bin]**

Id 0 selects directory rows whose edition contains that text and whose `lastwrite` is under 10 minutes. The row columns are name, language, description, homepage, version, player count, address and port. **[bin]**

Id 1 loads the account by name. The password is not placed in the query. The stored value is a `crypt` hash of the first 8 characters, with a two-character salt taken from the usual 64-character alphabet. The check is local. **[bin]**

What the client is shown

A hard failure (unknown account, wrong password, ban) is message id 4, the error text, and the link closes. The texts include `The password is wrong`, `Your account is currently banned.`, and `The account … doesn't exist yet, please sign up for it first.` **[bin]**

A success is message id 0 (the directory line) and id 2 (the status text), and then the link closes. Status text can be `Welcome to Graal!`, a staff sentence (global, or limited to named servers), a remaining-time sentence (years, days, hours), the trial sentence, or the free-server sentence (`Thanks for buying Graal! You will have no limitations on the` / `free servers`…). **[bin]**

The trial sentence and the free-server sentence also send id 3 first, carrying the `showmore` URL. That is a URL the operator configured. The process does not open a browser and does not read a saved login. **[bin]**

What a game server sends

The first line is the login. Its first byte is kind 0 or 1 (`byte - 32`). The next 8 bytes must be `GLSER001` or `GLSER002`. The rest of the line is the world name. Any other kind or token closes the link. Kind 1 suppresses the ghost queue message below. The world name is looked up, and the socket address must match the address stored for that world, or the console says the server is not allowed to connect from its address. **[bin]**

After login, only two ids are acted on:

Id	Meaning
4	Player add or update. Two 7-bit bytes of player id, a flag byte, a length byte, then the name. Bit 0 and bit 1 of the flag are stored. If bit 1 is set and the login kind was 0, the listserver replies to the game server with id 7: the coded player id and, when the player is queued, `N ghost(s) before you`.
5	Player leave. Two 7-bit bytes of player id. The player is removed and the limit is checked again.
other	Ignored.

The public directory is not built from these messages. It is the MySQL `serverlist` table. The game-server link is presence, the ghost queue, and the address check. **[bin]**

Account columns read for a client login: account name, encrypted password, email, blocked, only-load, admin level, admin worlds, ban time, ban reason, registration end. **[bin]**

What it never sends

No registry walk, no saved-login harvest, no browser, no plaintext password in the account query, no gen-5 list handshake. **[bin]**

`tools/classic-tools/text-convert.md`

TextConvert

Date: 2026-11-01

Delphi window, about 300 KB, no socket import. The readme is three lines: on start it converts every level in the current folder and in subfolders into New World levels, including links. **[doc][bin]**

The image imports `FindFirstFileA` / `FindNextFileA`, so the walk is a directory walk, which matches the readme. It contains the level signatures `GR-V1.00`, `GR-V1.01`, `GR-V1.02`, `GR-V1.03` and `GLEVNW01`, and the section words `BOARD`, `LINK`, `SIGN` and `NPC`. Failure strings include `Wrong link data!`, `Wrong chest data!`, `Decompression error!` and `File does not exist`. **[bin]**

A few strings (`Server error`, `Do you really want to quit Graal?`, `start1.graal`) come from the shared editor unit compiled into the tool. The import table has no socket library, so those strings are not a connection. **[bin]**

It does not log in, does not ask a listserver for anything, and does not send a level anywhere. Output stays on disk as the converted levels. **[bin][doc]**

`../level-convert/` is a later converter for the whole level chain (Z3-V1, GR-V1, GLEVNW01, a gmap, and a future package). TextConvert itself, as read above, takes GR-V1 levels from a folder and writes GLEVNW01. **[bin][doc]**

`tools/generators-222/README.md`

Graal 2.22 generators

Date: 2026-11-01

The `generators` folder in the Graal 2.22 tree holds three writers. Each one below is a Python 3 reference (standard library only) of the file it writes. The windows are not reproduced. The original executables were not run. Static reads of the Linux dungeon binary and the terrain and level-generator images supplied the file layout, the colour table, and the forest stamp. Tags: **[doc]** the text in that folder, **[bin]** a static read, **[file]** a small options or script file from the same folder.

`seihai-core` already parses `GLEVNW01` boards, links and NPCs (`assets/level.rs`) and a `GRMAP001` `LEVELNAMES` grid (`gmap.rs`). It skips generator keys (`GENERATED`, `HEIGHTMAP`, and the rest) and does not keep a text `HEIGHTS` block. These references stay in Graal_analysis.

Module	Tool in the 2.22 tree	What the reference writes
`terrain.py`	`TerrainGenerator.exe` (2003-12-16)	Plasma control grid → `GRMAP001` generator form and `GLEVNW01` levels of tile 0 plus a 9×9 `HEIGHTS` block and edge `LINK`s.
`dungeon.py`	`dungeongen.exe` and the Linux `dungeongen` (2003-05-23)	`GDUNG001` options, an `X/.` layout or a named style, 8×11 blocks, edge links, NPCs, short `GRMAP001`.
`levelgen.py`	`GraalLevelGen_nw-gmap.exe` (2003-03-07)	Twelve colours, a 6-pixel grid, forest stamp, `GLEVNW01` and a `LEVELNAMES` gmap. A second run paints over an existing level.

`nwtext.py` is the shared text writer. Tests build their own grids and pixels.

Not reimplemented

- **gif2mng.exe**, in the client tree next to `generators/`. The frame rule in `../gif2mng.md` is implemented in `../level-tools-gaps/gif2mng.py` (one frame PNG, more frames MNG, `-b` forces transparent index 2 on the existing palette). The seven slot RGB values are not in that note, so the writer does not invent them.
- **GraalEditor** and **GraalShop**. Editors, not generators. See `../graaleditor.md` and `../graaleditor-vs-205.md`.
- **TextConvert**, **GChar**, and **Graal Character Designer**. Not in this folder. See `../classic-tools/`.
- **GraalLevelGen_graal-txt.exe** (2004-01-18) and **GraalLevelGen_nw-txt.exe** (2001-06-30). File names and `bigmap.txt` are in `../level-tools-gaps/`. This module stays the nw-gmap build, which matches `levelgendoc.html` from the same day. The two txt programs do not write that gmap.
- The nw-gmap image can also write `GR-V1.03` when the output name does not end in `.nw`. **[bin]** This module writes the `.nw` branch only.
- DungeonMaker 0.9's crawlers. The Linux `dungeongen` links that library for the random styles and reads `layouts/rooms*`, `layouts/stats*` and `layouts/constants`. **[bin]** The style functions here draw the shapes named in `docu_dungeongenerator.txt` (maze, onion, spiral, rogue, snake, square spiral). They are not that library. The manual names the styles and the option bits and does not give the crawler steps, so `../level-tools-gaps/` does not add a second generator.
- Water, mountain and sand edge pieces in GraalLevelGen. The image tests neighbour flags and blits small stamps. **[bin]** The March 2003 manual names the colours and not the edge tiles, so this reference still marks those colours with one fill tile. `../level-tools-gaps/` records that limit.

Terrain

Dialog fields are grid width and height, base, height, chaos, even borders, an islands preset, and a seed. **[doc][bin]** Islands are even borders: the rim of the control grid stays at the base. The gmap keys, in the order the image's literal pool lists them, are `WIDTH`, `HEIGHT`, `GENERATED 1`, `GENSEED`, `GENBASE`, `GENEVENBORDERS`, `GENHEIGHT`, `GENCHAOS`, `LEVHEIGHT`, `LEVCHAOS`, `MAPIMG`, one level name per line, `HEIGHTMAP`, `RANDOMSEEDS`. **[bin]** Level names are the world name, an `A`…`Z`/`AA` column, and a zero-padded row, plus `.nw`. A level is `GLEVNW01`, 64 `BOARD` rows of tile 0 (`64 0` and 128 `A`s), `HEIGHTS` (nine lines of nine numbers), and a `LINK` on each edge that has a neighbour. West arrives at 61, east at 0, north at 61, south at 0; the other axis is `playerx` or `playery`. **[bin]**

The displacement is the usual midpoint split. The random step is the Delphi generator (`seed * 134775813 + 1` in 32 bits). `LEVHEIGHT` and `LEVCHAOS` default to the map height and chaos. The numbers will not match a save from the window.

Dungeon

`dungeonoptions.txt` starts with `GDUNG001`. **[file]** A `LAYOUTFILE` of `X` (wall) and `.` (path) wins over `LAYOUTSTYLE`. Each cell is 8 tiles wide and 11 tall, then cut into 64×64 levels. **[doc]** Level names are `LEVELSTART`, a column letter, and a 1-based row (`cave_A1.nw`). The gmap is three lines after the signature: `WIDTH`, `HEIGHT`, `GENERATED` plus the bottom-right name. **[bin]**

Place names, for NPCs and for `TILES` blocks of 88 ids:

- `path` — the cell is not `X`.
- `walln` / `walls` / `wallw` / `walle` — exactly one open side, and the wall sits on the opposite side (`walln` has path to the south, the north wall of a room). That matches the torch lines in the shipped options. **[doc][file]**

- `wallnw`, `wallne`, `wallsw`, `wallse` — no open side, and exactly one open diagonal. A rectangular room's northwest corner has its path on the southeast diagonal.
- `wallnw2`, `wallne2`, `wallsw2`, `wallse2` — two adjacent open sides (a 270° notch).
- `walld1` — path to the north and the south. `walld2` — path to the east and the west. Those two names are in the image and in the example `TILES` lines, not in the manual's place list. **[bin][file]**
- `wall` — anything else, including a solid cell.

Without a `TILES` line the reference uses a plain block: path is tile 0, a wall is tile 1 with the open side cleared. The original's default sheet is the cave png, which this does not redraw. **[doc]**

`NPC` lines take `image`, `script`, `blocking`, `underplayer`, `dominant`, `chance`, `place`, `x`, `y`. Chance is 1 to 100, with or without a `%`. `x` and `y` are offsets inside the 8×11 block. A false `blocking` adds `dontblock`; `underplayer` adds `drawunderplayer`. **[bin]** `ADDTESTNPCS` adds one NPC per level whose script is the `addtiledef` / `addtiledef2` / `loadmap` test from the manual. **[doc][bin]**

`LAYOUTSYMMETRY` 1 leaves the grid alone, 2 mirrors across the vertical centre, 3 across the horizontal centre, 4 does both, and 0 picks one of 1–4 from the seed. `DOUBLE 1` draws a maze on half the cells and scales it up; `DOUBLE 0` carves at full size. `LAYOUTORIENTATION` is accepted and stored; the sample file sets it to 1 and the reference does not rotate.

Level generator

Draw a picture whose width is 64 times the levels across and whose height is 64 times the levels down. Colours, with the match limit of 32 from the image: **[doc][bin]**

Index	Name	RGB
0	Grass, not drawn	24, 140, 24
1	Forest	181, 239, 189
2	Water	0, 0, 255
3	Mountains	128, 0, 0
4	Flowers	255, 0, 0
5	Sand	150, 110, 0
6	Puddle	0, 255, 255
7	Puddle stone	0, 179, 179
8	Small sand stones	113, 82, 0
9	Big sand stone	83, 61, 0
10	Swamp	0, 106, 0
11	Bushes	0, 66, 0

The grid is `(width + 5) // 6` by `(height + 5) // 6`. Cell `(x, y)` reads the pixel at `(x * 6, y * 6)`. **[bin]** Forest blits this 6×5 stamp at `(x * 6, y * 6 + 1)`, shifted 3 tiles left on odd rows:

```
25 26 27 28 29 30
41 42 43 44 45 46
57 58 59 60 61 62
73 74 75 76 77 78
89 90 91 92 93 94
```

Other matched colours write one fill tile (water 12, mountains 80, flowers 4, sand 5, puddle 6, puddle stone 7, small stones 8, big stone 9, swamp 10, bushes 11). An unknown colour, and grass, write nothing.

Level names are `world_a-01.nw` through `world_z-99.nw` style: a lower-case column, a dash, and a 1-based row of two digits. **[doc]** The gmap is `WIDTH`, `HEIGHT`, and quoted `LEVELNAMES` rows. **[bin]** Edge links use the same 0/61 arrival as the terrain levels. A second run keeps existing `LINK` and `NPC` lines and paints only the new non-grass tiles. **[doc]**

`tools/generators.md`

Generators

Three writers emit `GLEVNW01` / `GRMAP001`. They do not emit the same file. The Graal 2.22 programs are reimplemented in `generators-222/`.

TerrainGenerator

Plasma midpoint displacement. Writes:

- `GRMAP001` with `GENERATED`, `GENSEED`, `GENBASE`, `HEIGHTMAP`, `RANDOMSEEDS`, and per-level `HEIGHTS` blocks (9×9).
- `GLEVNW01` levels that are 64 rows of tile 0 plus a `HEIGHTS` overlay and edge `LINK` lines. No NPC section.

DungeonGenerator

8×11 stamp layout (hand-drawn, style, or DungeonMaker 0.9). Writes:

- One base64 `BOARD` of 64×64, edge `LINK`s, and `NPC` blocks.
- Stamp id 0 is replaced by a 4×4 ground fill before storage. Stamp id 1 is a hole and is not written.
- Short-form `GRMAP001`: `WIDTH`, `HEIGHT`, `GENERATED` (last level name).

GraalLevelGen

Image-to-linked-levels pipeline. Emits `LEVELNAMES` links. A third writer of the same signatures.

The 2004 graal-txt build and the 2001 nw-txt build write `bigmap.txt` instead of that gmap. Their file names and level containers are in `level-tools-gaps/`. The March 2003 manual does not name water, mountain or sand edge tiles, and `docu_dungeongenerator.txt` does not give DungeonMaker 0.9's crawler steps. Both limits are in that note.

Do not feed one writer’s output to another’s loader and expect a playable board.

`tools/ghidra/README.md`

Ghidra headless salvage pipeline

Reference scripts for static reads of recovered server tools, staff clients, and trainers. They live here because the exporter writes decompiler output, import lists, and addresses into a local folder. That output is never committed. Hand-written plain-behaviour summaries of those binaries live in `squirelworks/seihai-servertools` (`recovered-drive-tools/`, `recovered-hacks/`), not in this repository and not in the Seihai client.

Treat inputs as recovered tooling. Record purpose, CLI, protocols, and config roles in your own words. Do not invent malware or exploit payloads.

Layout

Path	Role
`current → ghidra_12.2_PUBLIC/`	Installed Ghidra release (gitignored)
`ghidra_scripts/ExportSalvageArtifacts.java`	Post-analysis export script
`analyze_tool.sh`	One-shot wrapper: prelim snapshot + headless. Raw files stay in `local-out/`.
`projects/`	Scratch Ghidra projects (deleted per run by default)

`analyze_tool.sh` reads `JDK_HOME` or `JAVA_HOME` (JDK 25 or newer). Optional `RIZIN_BIN` is a directory prepended to `PATH` when `rz-bin` is not already there. There is no default install path.

The second argument of `analyze_tool.sh` is the directory for a hand-written `SUMMARY.md`. The script creates a counts-only stub there if the file is missing, and leaves an existing summary alone. For the recovered Drive tools and trainers, that directory is the matching folder already kept in seihai-servertools. Do not commit a second copy of those notes here.

Raw exporter output (strings, imports, decompilation, disassembly) goes to gitignored `local-out/` and is not committed.

Versions

See `VERSION.txt`.

- **Ghidra:** 12.2 PUBLIC (`20261016`)
- **JDK:** Temurin 25.0.5+8 (Ghidra 12.2 requires Java 25+)
- **Helpers:** `file`, `strings`, `binutils` (`objdump`/`readelf`/`nm`), `radare2`/`r2`, `rizin`/`rz-bin`, `xxd`, `python3`

Install / refresh Ghidra

If `current/` is missing (fresh clone), from this directory:

```
mkdir -p /tmp/ghidra-dl
cd /tmp/ghidra-dl
curl -L -o ghidra.zip \
  "https://github.com/NationalSecurityAgency/ghidra/releases/download/Ghidra_12.2_build/
ghidra_12.2_PUBLIC_20261016.zip"
unzip -q ghidra.zip
mv ghidra_12.2_PUBLIC /path/to/Graal_analysis/tools/ghidra/
ln -sfn ghidra_12.2_PUBLIC /path/to/Graal_analysis/tools/ghidra/current
```

Install a JDK 25 build yourself and export `JDK_HOME` to it. Point Ghidra at that JDK by setting `JAVA_HOME_OVERRIDE` in `ghidra_12.2_PUBLIC/support/launch.properties` to the same directory.

Verify:

```
export JDK_HOME=/path/to/jdk-25
"$JDK_HOME/bin/java" -version
tools/ghidra/current/support/analyzeHeadless 2>&1 | head
```

Analyze one binary

```
export JDK_HOME=/path/to/jdk-25
tools/ghidra/analyze_tool.sh \
  /path/to/binary \
  /path/to/summary-dir
```

Optional third arg: max functions to decompile (default `40`).

```
tools/ghidra/analyze_tool.sh ./some-tool /path/to/summary-dir 60
```

Output tree

```
summary-dir/
  SUMMARY.md                # hand-written prose only (the only note worth keeping)

tools/ghidra/local-out/<tool>/    # gitignored
  summary_stats.txt         # counts; string bodies omitted
  functions.txt|.csv
  imports.txt
  exports.txt
  strings_interesting.txt
  decompiled/
  prelim/                   # file/strings/readelf/rizin/radare2
  logs/
  DO_NOT_COMMIT.txt
```

Do not copy `local-out/` into `SUMMARY.md`. Describe behaviour in your own words. Do not write dotted addresses, decompiled C, or auto-generated function names into a committed note. Labs stay on the loopback address.

Raw analyzeHeadless

```
export JDK_HOME=/path/to/jdk-25
export JAVA_HOME="$JDK_HOME"
export GHIDRA_HEADLESS_MAXMEM=4G
GHIDRA=tools/ghidra/current
$GHIDRA/support/analyzeHeadless \
  tools/ghidra/projects myproj \
  -import /path/to/bin \
  -overwrite -deleteProject \
  -scriptPath tools/ghidra/ghidra_scripts \
  -postScript ExportSalvageArtifacts.java /tmp/out 40
```

Point `-postScript` at a gitignored directory (`local-out/` or `/tmp`). Do not point it at a path whose text files would be committed.

Notes

- Headless deletes the scratch project by default (`-deleteProject`) so reruns stay clean.
- Decompilation targets entry points, name-matched helpers, then large or busy functions. That C stays under `local-out/`.

- The exporter skips unnamed auto-generated functions when it builds the local counts file, so those names are not copied forward.
- Interesting-string filters bias toward paths, CLI flags, URLs, config extensions, and server-related tokens. Those hits stay under `local-out/` and are not written into `SUMMARY.md`.
- `summary_stats.txt` records counts only. It does not embed string bodies.
- Committed summaries are prose: what the tool does, which screens or formats it touches, and what a later clean-room note could learn. No string dumps, import lists, or addresses.

`tools/gif2mng.md`

gif2mng

GIF to PNG or MNG. Banner date 2004-06-03. PE32, base 0x400000, entry 0x416558. Units `UGif` and `UMNG`, zlib 1.1.2.

- One frame → PNG.
- Two or more frames → MNG.
- `-b` snaps the palette to the seven Graal body slots and forces transparent index 2.

The converter does not invent a second palette; it rewrites the existing one to the body slots.

`level-tools-gaps/gif2mng.py` writes the frame rule above on synthetic frames. The seven slot RGB values are not listed here, so `-b` forces transparent index 2 and leaves the existing RGB entries unchanged. TERM's delay fields are not listed here either, so the writer does not emit TERM.

`tools/graaleditor-vs-205.md`

GraalEditor and GraalShop compared with the 2.05 built-in editor

Date: 2026-10-21

Sources: forms, RTTI class lists, version resources, and strings of `GraalEditor.exe` 2.3.1.1 (from a memory image of the unpacked Thinstall package), `GraalShop.exe` 1.1.1.0, and `GraalShop_old.exe`, all from the Graal 2.2.2.0 folder. The 2.05 side is in `client-internals/level-editor-205.md`. For GraalEditor on its own (menus, options, file types), see `tools/graaleditor.md`; this note only covers what changed relative to 2.05. Names only, no code. Tags: **[GE]** GraalEditor · **[GS]** GraalShop · **[2.05]** · **[inf]**.

Lineage

Build	Editor
2.05 (2001)	Built into `Graal.exe`. F4 switches the game window into editor mode. Refused while online.
2.17 / 2.22	Still built in, with the newer forms (`TEditNPC`, `TTilesArrayFrame`; 2.22 also `TLogFrame`).
2.31	No editor forms left in the client (only the 9 tutorial levels remain). The start screen has an **Editor** button that launches `GraalEditor.exe` from the game folder (seen live).
GraalEditor 2.3.1.1 (2005)	The same `TAdventure` program with the online client removed, shipped as its own exe in a Thinstall package.

GraalEditor is not a rewrite. It keeps the 2.05 main form (`TAdventure`, caption `Graal`), the same tool forms, the same toolbar hints ("Undo board copy", "Turn selection into a warp field"), and even game-only strings ("You cannot open the editor in the online mode.", F7 Player List and F8 Server List buttons in the help form). It also keeps the whole game model (`TPlayer`, `TComputer`, `TAdventureCommand` … the GS1 interpreter), so *Play* still runs a level locally. **[GE]**

It does not have the network client. There is no `TGraalClient`, `TAdventureClient`, or `TServerListClient`, no listserver host, and no login token. **[GE]**

What GraalEditor adds over 2.05

Area	2.05 built-in	GraalEditor 2.3.1.1
Open	GR-V1.00–1.03, Z3-V1.03/1.04, GLEVNW01	Same set (`Graal levels (*.graal, *.nw)` filter)
Save	`.nw` → GLEVNW01, otherwise GR-V1.03	GLEVNW01 with a new `HEIGHTS` section. GR-V1 writer strings are still present. **[inf]** "Level(s) saved" means sever
World	single level plus *Generate map* image. 2.05 already contains a `GRMAP001` reader (`WIDTH`, `HEIGHT`, `GENERATED`, `GENSEED`, …, `HEIGHTMAP`), but no caller was found, so it looks dormant. **[2.05][inf]**	**gmaps** wired into the editor: the same `GRMAP001` reader plus `LEVELNAMES`, `MAPIMG`, `MINIMAPIMG`, `NOAUTOMAPPING`, (`GENSEED`, `GENBASE`, `GENEVENBORDERS`, `GENHEIGHT`, `GENCHAOS`, `LEVHEIGHT`, `LEVCHAOS`, `HEIGHTMAP…HEIGHTMAPE` `RANDOMSEEDSEND`). A level cache (`TEditorLevelCache`) holds neighbours.
Terrain	tile overlays in the editor. The dormant gmap parser knows `HEIGHTMAP`, and `HEIGHTS` appears in the binary, but the editor has no height tools.	`TLevelHeights` height overlay per level (`HEIGHTS`, `.height`, "Terrain Height Spot", "Show terrain grid")
Palette modes	predefined object, tile, baddies/NPCs/chests	the same three plus **Predefined NPCs** (`TEditorDefNPC`, "Editor Control-NPC" with a parameter grid)
NPC editing	`TEditPerson` (image + *Action*) → `TPersonAction` (script)	one `TEditNPC` window with image, script, Test, Style, and **Find**. Scripts may carry editor markers `//#EDIT` and `//#`
Character NPC	one-page form	the same fields split over four tabs (Look, Movement, Attack, General)

Area	2.05 built-in	GraalEditor 2.3.1.1
Tiles	—	*Export Tiles Array* (`TTilesArrayFrame`): the selection as one line of tile indices and as a script array
View	blocking/foreground/terrain overlays	plus *Hide npcs* and *Show terrain grid*
Log	—	`TLogFrame` (memo log)
Offline data	13 embedded `tutorial_*` levels (2.05 also reads `tiledefsOffline.txt` offline)	`offlinetutorial_*` (3×3) and `tiledefsOffline.txt`
Sound	DelphiX + MCI/Winamp	BASS, with the same function set the 2.22 client imports
Integrity	`TProgramCheck` self-check ("The Graal execution file is corrupted…")	the same, plus RSA/SHA-1 signature checks of `GraalEditor.sig` and `arrays2.sig` (`TAS_RSACryptography`, `TA` verify signature, must have valid public key"). Live, the editor starts normally with no `GraalEditor.sig` and with a check does not gate startup.

Unchanged from 2.05: the tile undo model (rectangle snapshots, "Undo board copy"), the link, sign, chest, and baddy dialogs, GOBJSET01 stamp sets with `objects<set>.txt`, right-click to copy objects, and *Generate map* → `graalmap.png`.

GraalShop (the GANI editor)

`GraalShop.exe` 1.1.1.0 (version resource: "Gani Editor", 2006) and `GraalShop_old.exe` (2002) are separate Delphi programs with one main form, `TEditFrame` ("Graal Shop - The Graal Animation Editor"), and an *Add sprite* dialog. They edit `.gani` files only. Levels are out of scope. **[GS]**

Model

`TGraalAni` holds sprites (`TGraalAniSprite`, plus `TGraalSpriteAttachment` for `ATTACHSPRITE`/`ATTACHSPRITE2`), frames (`TGraalAniStep` → `TGraalAniPart` placements and `TGraalAniSound`), and, in movie mode, actors (`TGraalActor`, `TGraalActorMod`, `TActorSingleMod`). Undo and redo are a list of `TEditOperation` records.

Workflow

- **Sprite mode** (`GANI0001`): a direction selector (up/left/down/right), a sprite-definition grid, a frame strip, and a canvas. Toolbar actions: play/stop, add/delete/copy frame, insert step before/after, add sound, add/import/remove-unused sprite definitions, reverse animation, undo/redo, new/load/save/save as. Selected sprites can be moved, brought to front, or sent to back, and keyed with x/y.
- **Movie mode** (`GANI0002`, *-> Movie Mode*): a timeline of actors of kind CHAR, SPRITE, SOUND, or TEXT. TEXT actors have font, zoom, size, alignment, wordwrap, and shading. CHAR actors have ani, dir,

head, body, sword, shield, horse, colours, attr, and param. Options: *Automatic Gani Setting* (switch idle/walk when an actor is moved) and *Graal2002 ganis*.
- **Header fields** edited in a grid: Length, Loop, Back. color, Setbackto, Def. head, Def. body, Def. param1–3. These are written as `LOOP`, `BACKCOLOR`, `SETBACKTO`, `DEFAULTHEAD`, `DEFAULTBODY`, `DEFAULTPARAM`, `DEFAULTATTR`, plus `COLOREFFECT` / `ROTATEEFFECT` / `STRETCHXEFFECT` / `STRETCHYEFFECT` / `ZOOMEFFECT` and `SCRIPT`…`SCRIPTEND` (copied, not run). The grammar is in `formats/gani.txt`.
- **Preview** uses DelphiX DirectDraw 7 and plays `PLAYSOUND` steps through DirectSound. Default art: `sprites.png`, `body.png`, `head0.png`, `shield1.png`, `ride.png`, `soundicon.png`. The program finds the Graal folder through `Graal.exe` and `FILENAMECACHE.txt` and searches `pms`, `profiles`, and `weblevels`.

`GraalShop_old` has sprite mode only. It has no `GANI0002`, `MOVIE`, `ACTOR`, `ATTACHSPRITE2`, or `ROTATEEFFECT`. **[GS]**

How this compares with what 2.05 can play

The 2.05 client reads only the sprite-mode subset: `SPRITE`, `ANI`…`ANIEND`, `WAIT`, `PLAYSOUND`, `LOOP`, `CONTINUOUS`, `SETBACKTO`, `SINGLEDIRECTION`, `COLOREFFECT`, `ZOOMEFFECT`. It has no `GANI0001`/`GANI0002` signatures, no movie mode, no `DEFAULT*`, and no `ATTACHSPRITE*`. **[2.05]** Ganis saved by the 2006 GraalShop with movie mode or attachments do not play in 2.05. A sprite-mode gani with only the fields above does.

`tools/graaleditor.md`

GraalEditor

Version 2.3.1.1, Linux Cyberjoueur. From 2.3 the editor and the online client are separate executables.

Packing

The copy shipped in the client tree is a Thinstall package (pre-VMware ThinApp). Magic `0x82D0B188`, LoaderFlags reused as an offset. The Delphi image is recovered by running the stub decompressor and the `0xD7` block codec. PE version resource (present even in the stub): CompanyName `Linux Cyberjoueur`, FileVersion `2.3.1.1`, InternalName `Graal editor`, ProductName `GraalEditor`. Linker 2.25.

See also `tools/level-editor/` for the built-in editor of each client (2.22 as the reference). The decimal-`BOARD 0` claim in the next section conflicts with the base64 rows that the 2.05 and 2.22 built-in editors write; it has not been re-checked against a file written by GraalEditor itself.

Document model

Main form `TAdventure`, document `TAdventureLevel`, unit `UAdventureEditor`.

Sections the editor reads and writes (field order follows the dialogs, not the client loader):

- `BOARD 0` — 64 lines of 64 decimal tile indices (not base64 runs).
- `LINK`, `SIGN`, `CHEST`, `NPC`, `BADDY`, `LEVELNAMES`.
- Heights kept as a `TLevelHeights` overlay, not a second BOARD.

A file of raw decimal indices does not pass the client `GLEVNW01` loader until rewritten as base64 runs. The reverse is also true: a client `.nw` is not a `BOARD 0` grid.

Collision

The editor does not store type bytes. Collision is decided later by the 4096-byte type table (wall ≥ 20, water == 11).

`tools/graalshop.md`

GraalShop

Delphi GANI editor. Name is a Photoshop pun, not a store.

Modes

- Sprite mode (`GANI0001`): four direction rows, placements on a sheet.
- Movie mode (`GANI0002`): actors of kinds CHAR, SPRITE, SOUND, TEXT.

Domain objects recovered from RTTI: `TGraalAni`, `TGraalAniSprite`, `TGraalAniStep`, `TGraalActor`, `TEditFrame`.

Contract

A GANI file is line-oriented. `SPRITE` / `ANI` / `ANIEND` / `PLAYSOUND` / `DEFAULTATTR` blocks. The stepper advances the frame index. The shop copies the script and does not run it.

Offline tiledef used by the client tree: `picso.png,offlinetutorial,1,0,0`.

`tools/lab-servers/README.md`

Lab servers: our gserver and listserver

Date: 2026-10-30. These are the two servers the offline lab runs (see `../lab-setup.md`). The prebuilt packs live in the project files at `seihai-lab/servers/`: `seihai-lab-servers.tar.gz` (GServer master `efb5b90`) and `seihai-lab-servers-beta4.tar.gz` (GServer beta4 with our patch); this folder versions the parts we wrote ourselves.

File	What it is
`listserver.py`	Our lab listserver, version 2 (2026-10-30). Python 3, standard library only.
`test_listserver.py`	Drives a running listserver with every login shape and a fake gserver. Exit 0 when all checks pass.
`run-lab.sh`	Starts the listserver and the prebuilt GServer on loopback; `stop` stops both.
`gs2lib-GNW03081.patch`	The one-line gs2lib change our efb5b90 gserver build carries (2.05 token in the 2.1 slot).
`gserver-beta4-lab.patch`	Our fixes to GServer-v2 beta4 `365bbd2`: three for real 2.05 and 2.17.1, one for a saved hidden state, three for 1.33 to 1.39 (apply in `server/`).
`gs2lib-CLVER_5_24.patch`	Adds 5.24 (`G3D15010`) to gs2lib as its own slot, `CLVER_5_24`, between 5.12 and 5.31x. Against gs2lib `5c36ab8` (what beta4 fetches).
`gs2lib-CLVER_5_24-b839106.patch`	The same change against gs2lib `b839106` (our efb5b90 build). Applies with or without `gs2lib-GNW03081.patch`.

File	What it is
`test_clver_5_24.cpp`	Checks the token → id → name lookups and the order 5.12 < 5.24 < 5.31x < 6.015 against a built gs2lib.
`clver-5-24-lab/`	The 2026-10-31 lab run of real 5.24r4 on stock and patched gs2lib: screenshots, client consoles, proxy lines for the file messages.
`run-lab-beta4.sh`	The beta4 pack's start script: listserver, the 2.x server on 14802 and the classic server on 14803.

Background on what each server is, and how they compare with GServer-v2 beta4 and the other open-source servers, is in `../../server-notes/lab-servers-2026-10-30.md`.

Our gserver

GServer-v2 **master at `efb5b90` (2025-01-07)**, built without the V8 NPC server and without UPnP, linked against gs2lib `b839106` with `gs2lib-GNW03081.patch`. The binary reports itself as `3.0.9-beta`. It is not beta4: the beta4 branch is 807 commits ahead of it.

The patch puts `GNW03081` (2.05) in gs2lib's 2.1 slot. Real 2.05 needs it on this build: in a pre-2.1 slot this build's pre-2.1 level sequence never sends `PLO_SETACTIVELEVEL` (156), and 2.05 waits on the loading screen for it. The patch drops 2.1's own token `GNW31101`, so a real 2.1 client cannot log in to this build; `lib-stock/` has the unpatched library.

Our beta4 gserver (2026-10-30)

GServer-v2 **beta4 `365bbd2`**, built here with GCC 14 (no V8, no UPnP), with beta4's own gs2lib `5c36ab8` unpatched. It reports `4.0.0-beta`. beta4 knows 2.05r2 (`GNW03081`) natively as `CLVER_2_052`, so 2.1 keeps its token. Stock beta4 refuses 2.05 and 2.17 at login because its `allowedversions.txt` only accepts 1.41r1 and 2.22 to 2.31; the pack widens that in config. With the versions accepted, beta4 still did not get real gen-3 clients into the game, and none of the causes is reachable from config (the player class comes from the login type byte). `gserver-beta4-lab.patch` fixes three things for them, plus one account-save fix:

1. **Login bundle remainder.** 2.05 and 2.17 send the login line and `PLI_LANGUAGE` in one frame. beta4's `parseLoginPacket` dropped the rest of that frame, so the gen-3 decrypt iterator ran one step behind for the whole session: 2.17.1 failed with "Invalid PlayerProp" and 2.05 never sent valid packets. The patch keeps the remainder and hands it to the new player.
2. **`PLO_SETACTIVELEVEL` (156)** is now sent to Original clients from 2.01 up, before the NPCs. 2.05 waits for it on the loading screen.
3. **`PLO_LEVELNAME` before the board** on the first load, as efb5b90 and the live-trace stub send it. After the board, 2.05 stays on the loading screen.

All three copy a packet order real 2.05 was already seen to accept; none adds behaviour.

1. **Never log in hidden.** The account loader clears the hidden bit (`STATUS` 0x02). From the v4 lab: a client that closed while hiding under a carried object was saved hidden and came back invisible until its first step. Not reproducible with 2.17.1, which sends its own status at login (an account saved with `STATUS 22` loads visible on both builds and is saved back as 20). Build steps are in the pack's `README.md`.

1.33 to 1.39 on beta4 (2026-10-30)

beta4's `generation` is server-wide and 1.x clients need `classic`, so the beta4 pack runs a second server folder, `servers/classic` ("Lab Classic", port 14803), beside the 2.x one. 1.33 to 1.39 have no listserver; they connect

straight to the loopback address on 14803 ("One Player Online"). Starting from the `../../client-internals/versions-133-139.md` live check, four more problems turned up:

1. **Id 25 kicked 1.33 to 1.37** (their weapon upload). Code: accepted and ignored; the weapon stays on the client.
2. **Prop 10 sent as a gani.** Code: a classic server sends the bow form.
3. **Kicked after about 30 s, all of 1.33 to 1.39.** beta4 took "packet count is 0" to mean "the next frame is a login", and `PLI_PACKETCOUNT` (31) resets the count. These clients send a lone id 31 every 10 s, so the next frame was parsed as a login and counted as invalid; the third one kicked them. Code: a login-done flag. This fits #119's "invalid packet count" warning and also covers any 2.x client that sends 31 alone.
4. **Zero max hearts.** Stock `(defaultaccount).txt` has `MAXHP 3.00`, which the loader reads as 0, on both servers. Config: `MAXHP 3`. The classic folder's default account also uses `head0.gif`, `sword1.gif` and `shield1.gif`, since the stock world has no images and these clients carry them only as `.gif`, and turns off the 2.x `-gr_movement` helper weapon.

Real clients: 1.33, 1.34, 1.35, 1.36, 1.37, 1.38 and 1.39 log in, show the right player and HUD, walk, and stay connected (no kicks or unknown packets); 2.05r2, 2.17.1 and 2.22 rechecked on the 2.x server. Under Wine these clients need `nodirectinput=true` in `graalconfig.txt` for keyboard input, and often start on their own pause menu (Continue/End), which P closes.

5.24 in gs2lib's version table (2026-10-31)

Stock gs2lib has no entry for `G3D15010` (5.24r3, 5.24r4, and 6.00 beta2, which sends the same token). GServer then gives 5.24 `CLVER_UNKNOWN` (0) and serves it every pre-2.1 file path, so the client saves PNG data as a file name and never gets files over 64000 bytes (`../../protocol/file-transfer.md`). Until now the lab got around this by renaming another build's table entry to `G3D15010`, which took that build off the table.

`gs2lib-CLVER_5_24.patch` gives 5.24 its own slot instead. It adds three lines: `CLVER_5_24` in the enum (`include/IUtil.h`), and `G3D15010` / `"5.24"` in `clientVersions[]` and `clientVersionString[]` (`src/IUtil.cpp`), all right after 5.12. The array index is the version id, so the enum and both arrays move together.

Version ranges. Every later id goes up by one, and 5.24 sorts between 5.12 and 5.31x. Every test above 2.31 in GServer efb5b90 and beta4 `365bbd2` compares against `CLVER_4_0211`, `CLVER_5_07`, `CLVER_5_12` or `CLVER_6_015`, so 5.24 takes the same branch in each as 5.30/5.31 do, and no other build changes branch. beta4's stock `allowedversions.txt` has `modern = G3D14097:G3D0511C` (5.12 to 6.037), compared by id, so 5.24 is now inside the stock modern range. Before the patch it was id 0 and outside it. A side effect of the stock check (`PlayerClient.cpp`, about line 396): a single entry that gs2lib does not know, such as a bare `G3D15010`, also resolves to 0, so it lets in every client whose token is unknown, not only 5.24. **[src]**

Rebuild GServer too. The `CLVER_*` constants are compiled into GServer, so a new `libgs2lib.so` under an old `gs2emu` shifts every comparison above 5.12 by one. For example, the old binary would treat 5.31x as 6.015. Always rebuild both together. For beta4 that happens anyway, because gs2lib is linked in statically.

Build and checks.

```
# efb5b90 (the lab pack): gs2lib b839106 + this patch (+ gs2lib-GNW03081.patch if 2.05 is wanted)
git -C dependencies/gs2lib apply gs2lib-CLVER_5_24-b839106.patch
cmake .. -DTESTS=OFF -DUPNP=OFF -DV8NPCSERVER=OFF -DSTATIC=OFF -DCMAKE_BUILD_TYPE=Release && make -j8
# beta4: apply gs2lib-CLVER_5_24.patch to the fetched gs2lib 5c36ab8 before building
g++ -std=c++23 -I<gs2lib>/include test_clver_5_24.cpp <gs2lib>/lib/libgs2lib.a -lz -lbz2 && ./a.out
# prints OK
```

Both gs2lib versions built, and `test_clver_5_24.cpp` printed OK on both. On `b839106` the program needs the `CLVER_WORLDS` line swapped for `CLVER_IPHONE_BELOTE_1_5`, because that version has no Worlds entry.

Lab run (real 5.24r4, GServer efb5b90, Wine 9.0, loopback-only namespace). This used the same server config twice, with `G3D15010` allowed and the S→C 178 warp from `v5x/lab/rules.py`. The only difference was the library and the binary. **[cap]**

	stock gs2lib `b839106`, prebuilt gs2emu	**`b839106` + `gs2lib-CLVER_5_24-b839106.patch`, rebuilt gs2emu**
`PLO_FILE` for `sword1.png`	`\x86*sword1.png‰PNG…` (no modtime)	`\x86&w6u\x84*sword1.png‰PNG…` (GINT5 modtime)
client console	`File download: 1.png‰PNG`, then `Couldn't open …/downloads/1.png‰png`	`File download: sword1.png (size: 7143) done`, same for `sprites.png`, `body.png`, `head0.png`, `shield1.png`, `idle.gani`
`pics1.png` (216,002 B)	S→C 30 `FILESENDFAILED` (over 64000 B for a "pre-2.14" client)	68 `LARGEFILESTART`, 84 size, seven 32000-B chunks, 69 `LARGEFILEEND`, "done"
screen after the warp	black: no tiles, no player	level drawn, player drawn with the downloaded head, body, sword and shield

Every file the patched run saved is byte-identical to the server's copy. Files under `clver-5-24-lab/`.

Not covered: 6.00 beta2 (same token, so it gets the 5.24 slot; not run) and 6.00 beta1 (`G3D03048`, still unknown to gs2lib). The beta4 build with the master patch was compiled and unit-checked but not run against a client.

Our listserver (version 2)

```
python3 listserver.py [LOGFILE] [--accounts FILE] [--name TEXT] [--host <loopback>]
```

Ports: 14900 for gservers, 14921, 14922 and 14923 for clients. Every client port accepts every login shape; the shape is read from the first message, not from the port.

Client sends	**Clients**	**Reply**
id 0 list name only, plain zlib (shape A)	2.00, 2.05	id 0 list, id 2 status, one plain zlib frame
id 0 + id 1 account/password, plain zlib (shape B)	2.17.1	the same, once
id 4 key + token, then bz2 frames (gen 4)	2.19 to 2.21	id 0 + id 2 in one gen-4 frame
id 7 key + token + list name, then gen-5 frames (shape C)	2.22, 2.31	two zlib (ctype 4) frames: id 0 list, then id 2 status + id 3 site URL + id 5 upgrade URL
id 5 key + token (RC list)	RC	as shape C, every server
a refused account (with `--accounts`)	any	id 4 with the error text, then the link closes

The gen-5 reply follows the public listserver's 14922 reply (`../../protocol/public-listserver-2026-10-25.md`), except that we do not add the two trailing NUL bytes the public server leaves after the last line.

Server names carry the list tag only for gen-4/gen-5 clients, from the gserver's `SERVERHQLEVEL`: 0 `U` (hidden), 1 `H` (bronze, the GServer default), 2 none (classic), 3 `P` (gold), 4 `3`. Classic clients always get the bare name, as on the public 14923 list.

Gserver side: `REGISTERV3`, `NEWSERVER`, `SERVERHQLEVEL`, `SETPLYR`/`PLYRADD`/`PLYRREM` (the players column is live), `VERIACC2` (answered `SUCCESS`, or the account error with `--accounts`), `SETIP` (beta4's 60-second keep-alive), `SENDTEXT` (logged), `PING` (a pong in GServer's naming; never answered). A gserver's row is dropped when its link closes.

What changed from the stub of 2026-10-22

The stub shipped in the lab pack (`listserver.py`, 119 lines) differed from the one that made the 2026-10-21 captures:

1. **No gen-5 login.** It handled id 4 (gen 4) but not id 7, so real 2.22/2.31 and Seihai's 2.22/2.31 profiles got no list: Seihai reported "the listserver sent no reply within 5 s". Fixed.
2. Shape B got the list twice (once for id 0, once for id 1). Fixed: once, after the account.
3. Rows were never removed, so a stopped gserver stayed listed. Fixed.
4. The players column was always 0. Fixed (PLYRADD/PLYRREM).
5. It answered gserver id 16 with id 99. Id 16 from GServer is a pong, and 99 is the listserver's ping, so a beta4 gserver would answer with another 16. GServer efb5b90 never sends 16 on its own, so this never looped in practice. Removed.
6. No RC (id 5) list, no account check, no site/upgrade URLs. Added.

Tests run (2026-10-30)

- `test_listserver.py` against version 2: all checks pass (shapes A, B, gen 4, gen 5, RC, gserver VERIACC2, player count, row dropped on disconnect, refused password with `--bad-login`). Against the old stub: 5 failures (gen 5, RC, duplicate reply, player count, stale row).
- Seihai `--list-only`, profiles 2.00, 2.05, 2.17, 2.22, 2.31: every profile lists the server with version 2. With the old stub, 2.22 and 2.31 fail.
- **Real clients under Wine** (loopback namespace, our gserver build), list then game login:
  - 2.05 rev 2: list on 14921 (shape A), in game on `onlinestartlocal.nw`.
  - 2.17.1: list on **14922** with shape B (its built-in port), "Classic" tab, in game.
  - 2.22: list on 14922 (shape C), "Bronze" tab, the two status lines, the Show More and Donations buttons; in game.
- **Real clients on patched beta4** (same namespace, listserver version 2, and again from the packed tarball for 2.17.1): 2.05 rev 2 (`GNW03081 (2.05r2)`), 2.17.1 (`GNW22122`) and 2.22 (`GNW03014`, logged as "New Client (2.22+)") each list the server, reach the game and walk; the gserver log shows no unknown or invalid packets. Unpatched beta4 with the widened `allowedversions.txt`: 2.05 stuck on loading, 2.17.1 "Invalid PlayerProp" on the first step. beta4 has no legacy port or old-client option (one listener, `serverport`); `generation = classic` makes 2.17.1 fail at login instead.
- `test_listserver.py` expects to be the only gserver on the listserver. Run it with GServer stopped; with GServer connected the row and tag checks fail because GServer's row comes first.

Notes for the other threads

- A v5/v6 client logs in to a gserver named "Login" on port 14900, which here is the listserver's gserver port. To use our gserver as that login server, move the listserver's gserver port (the `14900` in `main()` and `listport` in `serveroptions.txt`) and set GServer's `serverport=14900`.
- GServer knows `G3D22067` (5.07), `G3D14097` (5.12) and `G3D26090` (5.31x), not `G3D15010` (5.24); add unknown tokens to `config/allowedversions.txt`.
- `xtjoeytx/glauncher` redirects the official v4+ client by hooking `gethostbyname` and `connect`: `license.graal` line 1 is the host for `<login-host-4x>`, line 2 the port that replaces 14900, line 3 the host for `<graalonline-site>`, lines 6 and 7 `loginserver2`/`loginserver3`. In the lab a hosts file does the same job without a DLL.

`tools/lab-setup.md`

Offline lab for live captures

Date: 2026-10-21

This is how the 2.05 and 2.22 captures cited as **[cap]** in the protocol notes were made. Nothing here touches a public server.

Isolation

- **Network:** everything runs inside a Linux network namespace with only loopback up (`unshare --net --mount`, then `ip link set lo up`). The clients have no route anywhere else, and DNS fails outside the hosts file.
- **Hosts file:** a private one is bind-mounted over `/etc/hosts` inside the mount namespace. It points `<list-host-classic>`, `<list-host-reborn>`, `<list-host-graalcc>` and `<list-host-graalbiz>` at the loopback address.
- **Display:** each client gets its own Xvfb display, and Wine runs with a separate 32-bit prefix per client. Two copies of the same game folder let two clients run at once. Give them different `fixedudpport` values and set `noudp=false`, `fixedport=true`.
- **Capture:** `tcpdump -i lo` inside the namespace.

Game server

- GServer-v2 built without the V8 NPC server.
- Stock GServer does not know `GNW03081`.
  - Mapped to a pre-2.1 version slot, the login sequence never finishes: the client sits on the loading screen.
  - Putting `GNW03081` in the **2.1** slot of the client version table and adding it to `allowedversions.txt` lets 2.05 log in and play.
- 2.22 (`GNW03014`) works unchanged.

Listserver stub

A small script that speaks both sides: - **Server side, port 14900:** 1. Read the uncompressed `REGISTERV3` line, then zlib frames. 2. Record `NEWSERVER`. 3. Answer every `VERIACC2` with `SUCCESS`, echoing account, player id and type. 4. Answer ping. - **Client side, ports 14921/14922/14923:** - On id 0 (2.05 `newmain`) or id 1 (account/password), send id 0 with the server list and id 2 with a status line. - On id 7 (2.22), take the key and switch to post-2.2 framing.

2026-10-30: the stub shipped in the lab pack on 10-22 had lost the id 7 path, so 2.22 and 2.31 got no list from it. It is replaced by listserver version 2 in `lab-servers/` (gen 5, RC lists, live player counts, optional account file); see `lab-servers/README.md`. The 10-21 captures above were made with the earlier stub, which did answer id 7.

Lab accounts are throwaway names with dummy passwords. This lab's stub accepts any password (other stubs may check accounts), and the logs print account names only. Captures still contain the dummy passwords in clear, so they stay out of the repo.

Decoding

- **TCP:**
  1. Reassemble each direction.
  2. Split `u16be` frames and inflate (post-2.2 frames: undo the partial XOR first).
  3. Split lines on `\n`.
  4. For 2.05 client→server lines, replay the gen-3 iterator from the key in the login line and delete the byte at `(it & 0xFFFF) mod len(line)`.
- **UDP:** XOR each datagram with `recipientId XOR 1`, where the recipient is the player who owns the destination port. The first byte is then `seq`, the second the UDP id. See `protocol/udp-p2p.md`.

Capture set

The **[cap]** tags in the protocol notes come from five runs in this lab, all on 2026-10-21: 1. 2.05 listserver exchange and one 2.05 game login. 2. 2.05 and 2.22 game logins, and the 2.22 listserver handshake. 3. Two 2.05 clients in one level with UDP enabled. This is the source for the UDP key and payloads. 4. Two 2.05 clients and a bomb next to bushes. This is the source for the board change, item drops and respawn. 5. Three 2.05 clients in one level with UDP enabled (players 2, 3 and 4). This is the source for the per-recipient UDP key and the animation-letter field.

The pcaps stay on the lab machine. Only the decoded text, with credentials removed, is kept outside it.

`tools/level-convert/README.md`

Level conversion chain

One converter for the level generations, in this order:

`.zelda` (Z3-V1) → `.graal` (GR-V1) → `.nw` (GLEVNW01) → `.gmap` (a grid of level files) → a future `.seihai` package.

TextConvert (`../classic-tools/text-convert.md`) walks a folder and writes GLEVNW01 from GR-V1. It does not read Z3-V1, it does not write a gmap, and it has no package step. This tool is that chain, both directions, as far as each format can carry the data.

```
python3 tools/level-convert/convert.py <src> <dst>
```

The suffix chooses the format. Exit status is 1 when the file is truncated and 2 for any other refusal. Warnings go to stderr and the file is still written when the loss is only a field the target cannot store.

Binary levels are read with `formats/grv1-decoder/grv1.py`. A `.graal` board is written with `tools/file-formats/gwebl.py` `write_board` (13-bit GR-V1.03). Nothing here is a second board decoder. Tests are synthetic. No client program is run.

What is lossless

Path	What survives
`.graal ↔ .nw`	64×64 board (tile ids 0–4095), links (destination, box, `playerx` / `playery` / `playery+10` kept as written), signs, baddies with a numeric type, NPCs, chests whose item is one of the 25 names in `client-internals/level-editor-205.md`.

Path	What survives
`.zelda ↔ .graal ↔ .nw`	The same board and links when every tile is 0–2047, plus signs and numeric baddies. A real Z3-V1 file has no NPC section and no chest section, so those lists stay empty. They are not invented.
`.nw → .seihai → .nw`	The GLEVNW01 bytes.
`.gmap → .seihai → .gmap`	The GRMAP001 bytes.
directory → `.gmap` → directory	Width, height, and each level's file name and cell. The gmap does not contain the level bytes.

`.zelda` output is `Z3-V1.04` (the verse line is how baddy text is stored). `.graal` output is `GR-V1.03`, which is what the 2.05 and 2.22 editors write for a non-`.nw` file.

What is lossy

- **NPCs and chests into `.zelda`.** Z3-V1 has no NPC section and no chest section. The tool says so and writes neither. A file named `.zelda` whose magic is actually `GR-V1` is read by the magic (`formats/level-binary.md`); saving it as `.zelda` then drops those sections.
- **Tiles above 2047 into `.zelda`.** Z3-V1 codes are 12-bit. Those tiles are stored as the low 11 bits, and a warning names the count. `.graal` and `.nw` keep ids 0–4095.
- **HEIGHTS.** A 9×9 overlay is read and written on `.nw`. `GR-V1.04` is read as `GR-V1.03` plus that block. Saving `.graal` still writes `GR-V1.03` and says the overlay was not written. `Z3-V1.02` is read with 11-bit codes; saving `.zelda` still writes `Z3-V1.04`.
- **NPC coordinates that are not whole tiles.** `.nw` keeps `10.5`. GR-V1 stores a byte (`x+32`), so the fraction is cut off.
- **Link destinations with spaces.** GR-V1 keeps them (every token before the last six). GLEVNW01 keeps one token and the link is not written.
- **Baddy type names.** The notes give the display names and say the script names run from `graysoldier` to `dragon`, not the ten tokens. A numeric type 0–9 round-trips. A name is kept on `.nw` and is not given a made-up byte on `.graal` or `.zelda`.
- **`#K(13)` typed as sign text.** `grv1.py` drops that escape. A newline is stored as the editor does (`#K(13)` plus the newline code) and comes back as a newline.

A gmap is a directory

`formats/gmap.md` is the reader: `GRMAP001`, `WIDTH`, `HEIGHT`, `LEVELNAMES` as quoted CSV, then `MAPIMG`, `MINIMAPIMG`, `NOAUTOMAPPING`, `LOADFULLMAP`, `LOADATSTART`. Generator lines (`GENERATED`, `HEIGHTMAP`, …) are kept and are not turned into cells. `GENERATED` is not expanded.

Build one from a directory. Either `grid.txt` (this tool's index, not a client file):

```
WIDTH 2
HEIGHT 2
0 0 west.nw
1 0 east.nw
0 1 south.nw
```

or GraalLevelGen names `prefix_a-01.nw` (`tools/classic-tools/graal-level-gen.md`), column letter then a 1-based row. Holes are empty cells. Reading a `.gmap` writes `grid.txt` only. It does not create the level files, because the map does not hold them.

The converter refuses a grid above 512 on a side. That is this tool's own guard, not a client limit.

The package

`SEIHAI-FORMAT.md`. **[inf]** A 2.22 client does not read it.

`tools/level-convert/SEIHAI-FORMAT.md`

SEIHAI01 package

Date: 2026-11-01

Tag: **[inf]**. This is a future container for the Seihai toolchain. It is not a historical Graal format. A 2.22 client does not read it, and it is not in a client string table. Do not rename it to `.graal` or `.nw` and expect a client to open it.

The level generations stop at GLEVNW01 and GRMAP001. This wrapper only holds one of those files so a later tool can carry either without pretending the bytes are a new board encoding.

Layout

Text header, then the payload bytes exactly:

```
SEIHAI01
kind nw
bytes <N>
<N bytes>
```

`kind` is `nw` or `gmap`.

kind	Payload
`nw`	One GLEVNW01 level, the on-disk text from `formats/glevnw01.txt`.
`gmap`	One GRMAP001 map, the text from `formats/gmap.md`.

- The first line is the eight-character version `SEIHAI01`. A later container would use a new first line.
- `bytes` is the payload length in decimal. The payload starts on the byte after that line's newline. There is no compression and no extra checksum.
- A file shorter than the declared length is truncated. Bytes after the payload are rejected.
- `kind nw` must start with `GLEVNW01`. `kind gmap` must start with `GRMAP001`.

`tools/level-convert/convert.py` writes a level into `kind nw` (a `.graal` or `.zelda` source is converted to GLEVNW01 first, because that text can carry the NPC and chest sections a `.zelda` file does not have). It writes a directory or a `.gmap` into `kind gmap`. Reading the package back yields the same payload bytes.

`tools/level-editor/222-reference.md`

The 2.22 built-in level editor

Date: 2026-10-25. Primary reference. The 2.22 client carries the same editor as 2.17.1 (checked: same class list, same loader and writer shapes). Other builds are deltas in `version-deltas.md`. Tags are explained in `README.md`.

Table of contents: 1 Entering and leaving · 2 Window and tool modes · 3 Level model · 4 Loading and saving · 5 Tiles and the sheet · 6 Selection, paste, undo, fill · 7 Stamps · 8 Links · 9 Signs · 10 Chests · 11 Baddies · 12 NPCs · 13 Keys · 14 Tabs, maps and multi-level editing · 15 Other dialogs · 16 Open

1. Entering and leaving

The editor is the `TAdventure` main form in a second mode of the game screen. The same exe plays and edits. **[2.22]**

- **F4** opens the editor; it is refused online ("You cannot open the editor in the online mode."). The tutorial levels live inside the exe, so F4 from a tutorial level opens a new empty level instead of failing. **[prev][log]**
- **F5** opens the Load level dialog. **[prev]**
- **P / Play** returns to the game on the current level. Since 2.13 *Play* saves every modified level first. **[prev] [log]**
- A level path as the first command-line argument skips the login screen and opens that file in the editor. **[prev]**
- **F10** toggles the window border style (bordered / borderless). **[2.22]**
- **Esc** shows a warning box before leaving. **[2.22]**
- Dropping a text file onto the window creates an NPC from the script (1.41 feature, handler still present in 2.22). **[log]** The handler body is **[struct]**.

2. Window and tool modes

- **Board view**: a DirectDraw surface showing 64 × 64 tiles at 16 px, with horizontal and vertical scroll bars. Mouse wheel scrolls vertically; arrow keys scroll, also on single levels (2.15). **[prev][log]**
- **Toolbar**, in order: New, Open, Save, Save as, Undo board copy, Cut, Copy, Paste, Delete, Turn selection into a warp field, Edit links, Edit signs, a button that shows the current tile rectangle as text (added 2.16), Close, Play. **[2.22 for handlers present; labels prev]**
- **Tab bar**: one tab per open level. Close asks about unsaved changes. **[2.22]**
- **Side panel** has two radio modes plus a combo, giving these *palette modes*. **[prev]**
    1. Predefined object (stamp) from a GOBJSET01 set.
    2. Tile palette from the sheet.
    3. Baddies, NPCs and chests.
    4. Predefined NPCs (the `defnpc*.txt` scripts; change log v2.1, class present in the 2.1 test builds). A string grid edits the `#EDIT` parameters of the chosen script. **[log]**
- **Check boxes**: show blocking tiles, show foreground tiles, show terrain tiles. They are overlays computed from the 4096-byte type table, not stored. **[prev]**
- **Status bar**: mouse tile position and the tile under it (1.41). **[log]**
- Editor control-NPCs (since 2.13): a file `editnpc<Name>.txt` placed in the NPC folder scripts editor actions, with `mousex`, `mousey`, `mousescreenx`, `mousescreeny` and `mousebuttons` (left 1, middle 2, right 4) available only in editor mode. Only changing tiles is supported. **[log]**
- **M** builds a map image of the level or gmap (saved as PNG, not 8-bit). On gmaps it works since 2.16 rev 7. **[log][prev]**

Mouse handling (select, move, drag-drop, double-click by object kind) is a state machine over obfuscated fields. Only the outline is known: **[struct]**

- Left drag on empty board selects a tile rectangle, clipped to the level (fields outside the level cannot be selected; 1.41).
- With a tile chosen, left drag paints; pasting a floating block follows the mouse and a click outside drops it.

- Dropping a baddy, NPC or chest from the palette can be released and then moved with the mouse, like in the old days (2.13 rev 3 fix). **[log]**
- Right button on a placed baddy, NPC or chest clones it. **[prev]**
- Right button elsewhere is flood fill (see section 6). **[log]**
- Double-click on an object opens its dialog; double-click on a tile makes it the background tile. **[prev][log]**

3. Level model

`TAdventureLevel` owns these lists. **[2.22]**

List	Element	Notes
board	64 × 64 tile ids (16-bit)	index = y × 64 + x
links	`TEditorLink`	destination, x, y, w, h, new x text, new y text
signs	`TEditorSign`	x, y (tiles), message text
chests	`TEditorChest`	x, y, item, sign index
NPCs	`TEditorNPC`	image name or none, x, y (fractional tiles), script
baddies	`TComputer`	x, y, type, three verse strings
stamps	`TEditorObject`	only in the object set, not in the level
heights	`TLevelHeights`	9 × 9 floats, optional, edited only in builds with the height tools

The editor also keeps a cache of recently loaded levels (`TEditorLevelCache`, 2.17 and later) so that the link walker and the gmap code can open neighbours without a reload. **[2.22][prev]**

4. Loading and saving

Open

The open dialog filter is `Graal levels (*.graal, *.nw)`. Several files may be chosen and each opens in its own tab. **[2.22]**

The loader reads the first eight bytes and branches. **[2.22]**

Magic	Reader
`GLEVNW01`	text reader (below)
`GR-V1.00`, `GR-V1.01`, `GR-V1.02`, `GR-V1.03`	binary reader; 13-bit board codes for 1.02 and 1.03, 12-bit for 1.00 and 1.01
`Z3-V1.03`, `Z3-V1.04`	the old `.zelda` layout, 12-bit codes (accepted on open)
`GR-V1.04`	read only by the weblevels cache path in 2.22, not by the editor open path
anything else	rejected (the string table has "Unsupported file type!")

Section order and per-version content of the binary files are in `formats/level-binary.md` and `formats/level-binary-client.md`. The 2.22 reader keeps the same rules as 2.05: a baddy record is at least three bytes and the section ends at the first record shorter than three bytes **or whose first byte is 0x80 or more** (there is no x = 0 test). **[2.22]** This is the same shape in 1.39, 2.00, 2.05 and 2.17.1.

The GLEVNW01 text reader [2.22]

Lines are read until the end of the file. Block sections (`SIGN`, `NPC`, `BADDY`, `HEIGHTS`) swallow lines until their terminator; everything else is a one-line record split on spaces.

- **`BOARD x y w layer data`**
  - `x` and `y` must be 0–63, `w` ≥ 1, `x + w` ≤ 64.
  - The *layer* field (fourth) is **not read**. A row with a non-zero layer is still written onto the one board.
  - `data` needs at least `2 × w` characters. Each tile is two characters of the 64-letter base64 alphabet, high character first, so a tile id is 12-bit (0–4095).
  - A character outside the alphabet ends that row silently; tiles already placed from the row stay. Invalid rows are skipped; no error is raised.
  - Rows may come in any order and may be partial. The board is zero-filled first.
- **`LINK dest x y w h newx newy`**: at least five tokens (dest and four numbers). Missing `newx` / `newy` become `playerx` / `playery`. An empty value is also replaced with `playerx` / `playery`. The text is stored as given; arrival text such as `playery+10` is kept verbatim. **[2.22]** The same parse is in 2.05.
- **`SIGN x y`** then message lines until `SIGNEND`. The lines are joined with newlines and decoded through the sign alphabet (see `formats/sign-text.md`).
- **`CHEST x y item signindex`**
- **`NPC image x y`** then script lines until `NPCEND`. Image `-` means no image. x and y are decimal floats.
- **`BADDY x y type`** then exactly three verse lines, then `BADDYEND`. Position is clamped to 0–63; the type is bounded to 0–9. Extra lines before `BADDYEND` are ignored.
- **`HEIGHTS`** then up to nine lines until `HEIGHTSEND`. Each line is a comma list of up to nine decimals; line *i* is row *i* of a 9 × 9 grid. The block is read only when the level already has a height overlay. **[2.22]**

Save [2.22]

- **Save** (button and Ctrl+S) is a *save-all*: it first commits any floating paste, then walks every level in the editor's level list and saves each one that is marked modified, plus the current one. Levels without a file name are skipped. For each level it **deletes the old file and writes the new one in place**, using the writer picked by the extension, then clears the modified flag. If the map object is modified it is saved too. The status bar shows "N Level(s) saved" (and " + <map name>" when the map was written). **[2.22]**
- **Save as** asks for a name. Saving a level opened from a URL falls through to Save as. **[prev]**
- The **file extension picks the format**. `.nw` (case-insensitive) writes text; **any other extension writes `GR-V1.03`**. The editor never writes `Z3-V1`, `GR-V1.00`–`GR-V1.02` or `GR-V1.04` levels. **[2.22]** (The cache writer, which is not the editor save, writes `GWEBL001` and `GR-V1.03`/`GR-V1.04` for weblevels files.)

Text writer. `GLEVNW01`, then 64 rows `BOARD 0 y 64 0 <128 characters>`, then `HEIGHTS` + lines + `HEIGHTSEND` **only when the level has a height overlay marked modified**, then `LINK …`, `SIGN x y` + text + `SIGNEND`, `CHEST x y item sign`, `BADDY x y type` + three verses + `BADDYEND`, `NPC image x y` + script + `NPCEND`. **[2.22]** (2.05 writes the same without `HEIGHTS`.)

Binary writer (`GR-V1.03`). Header, board (13-bit run-length codes, including double runs `ABAB…`), links as text lines ending with a # line, baddies as three raw bytes (x, y, type, not +32) plus `verse1\verse2\verse3` and a newline, then `FF FF FF 0A`, then NPC records `x+32, y+32, image#script`, then a # line, chests as four GCHARs and a newline each, a # line, then signs as two GCHARs plus sign-alphabet text. Baddy types 4–6 write empty verses (`\\`). **[2.22]** Details of the run encoder match the 2.05 note.

Chests and baddies can be skipped. In both writers, chests and baddies are written only when two session-mode flags are both clear. If either is set the editor writes just the section terminators. The same flags gate the

corresponding part of the loaders. The flags are stored in an obfuscated form with a tamper check, so their meaning is **[struct]**. The New World betas have no chest or baddy editor at all, so the flags are most plausibly a *chests and baddies enabled* switch for a New World style mode **[inf]**.

Safety

2.22 deletes the old file and then writes the new one in place, so a crash between the two loses the level. The 2.3 change log says this was fixed there: the new level is written to a copy and renamed over the old one. **[2.22][log]**

5. Tiles and the sheet

- Level tile ids are 0–4095. The sheet `pics1.png` is 2048 × 512, 16-px tiles, 128 columns × 32 rows. **[2.22]**
- **Id → sheet position** (correcting `client-internals/client-assets.md`):

```
col = (id % 16) + (id / 512) * 16
row = (id / 16) % 32
source = (col * 16, row * 16, 16, 16)
```

The sheet is laid out as eight blocks of 512 ids; each block is 16 columns by 32 rows, filled row by row, and the blocks sit side by side. Ids 0–511 are the left 256 px, 512–1023 the next 256 px, and so on. The earlier note used `col = id % 128`, `row = id / 128`, which is wrong. Seihai's renderer already uses the formula above. - A *tile definition* (`tiledefs/…`, `addtiledef`) swaps the sheet for levels whose name starts with a prefix. Line format: `image,prefix,type,x,y`. Type 0 is the old order; type 1 is the new order (documented in the 2.1 change log). The sample file in the 2.22 install lists about 17 prefixed overrides (all `type` 4 for the extra sheets with x offsets such as 1344, 1536 or 1728) plus the default row `pics1.png,,0,0,0`. The meaning of type 4 and of the x/y values is **not traced**. **[inf]** They look like a tile-column offset into the replacement sheet. See `formats/tile-walk.md` for how types affect walking. - *Tile palette*: click picks one tile, drag from the upper-left to the lower-right picks a block (2.1 change log); the block pastes as a floating selection. Double-click a palette tile or a board tile to make it the *background tile* shown in a preview box (1.41). New levels start filled with the chosen background tile (1.41 fix). **[log][prev]** - Tile *type* (blocking, water, etc.) is not stored in the level. It comes from the 4096-byte type table (`formats/tiletypes.txt`).

6. Selection, paste, undo, fill

- **Selection** is a rectangle on the 64 × 64 board. Copy and Cut lift the rectangle into a floating block; Paste follows the mouse. Dropping writes tiles to the board. **[prev]**
- **Undo** covers board tiles only. Before each board write the editor stores the rectangle and the old tile values. The stack is per tab and holds at most **128** records. Undo restores the rectangle and re-selects it, and marks the level modified. Changes to links, signs, NPCs, baddies and chests are not undoable. **[2.22]**
- **Draw mode**: with a single tile selected, left-drag paints tiles until release (1.41). **[log]**
- **Flood fill** (change log v2.00): paste a tile or stamp, then press the right mouse button on the board to fill the area with it. Shift + right button fills only complete objects. With nothing selected, the right button fills with the background tile. **[log]** The handler is **[struct]**. (Right button over a baddy, NPC or chest clones the object instead.)
- **Turn selection into a warp field**: creates a new link covering the selection and opens the link dialog. The link defaults are **[2.22]**:
    - x, y = top-left tile of the selection, w, h = selection size; the destination is left for the dialog;
    - new x = `playerx`, new y = `playery`;

    - then one edge rule is applied, the first that matches: if the selection starts in column 63, new x = `0`; else if it starts in row 63, new y = `0`; else if it starts in column 0 and is one tile wide, new x = `61`; else if it starts in row 0 and is one tile high, new y = `61`.

  So a link on the east edge arrives at the left side of the next level, and a one-tile-wide link on the west edge arrives at x = 61, while the other coordinate follows the player. `playerx` / `playery` stand for "keep the player's own coordinate".
- **Selection to text** button: shows the current tile rectangle as text for use in editor NPCs (the `editnpc*.txt` scripts). Another button builds a tile-index line and a script array from the selection (`TTilesArrayFrame`). **[log][prev]**

7. Stamps (predefined objects)

- Sets come from `objects<set>.txt` in the tile-objects folder. The plain `objects.txt` is *Standard*. Format: `GOBJSET01` (`formats/gobjset01.md`). Five sets ship with the client (Standard, House Inside, Castle, Snow, Volcano; 288 named objects). **[prev]**
- *Add the selection to the objects* names the current rectangle and inserts it after the selected stamp; *Save objects* rewrites the set file. The name dialog is `TObjectName`. Its OK handler body is **[struct]**; the behaviour is the 2.05 one. **[prev]**
- Placing a stamp writes tile ids only (no layer, no type, no link). Collision is decided by the tile-type table.
- Object sets were added in 2.04 (change log v2.04; the `TObjectSet` class is first in the 2.04 exe). **[log]**

8. Links

- **Edit links** (key L) opens the link list: Linkfile, X, Y, W, H, New X, New Y. Buttons: Edit, Delete, Load. *Load* opens the destination in a new tab. **[prev][2.22]**
- **Link dialog** (`TLinkOptions`). Fields: file name or URL, x, y, w, h, new x, new y. OK validates the fields; on failure the dialog shows "Wrong values! Type in a filename…" and stays open. On success the link is added to the level list if it is not already in it, the list is refreshed, and the level is marked modified. **[2.22]**
- New x and new y may be a number or an expression: "you can use math operations and variables like playerx/y". The editor stores the text unchanged. Whether the engine evaluates an offset such as `playery+10` when the link fires was not traced. **[inf]** In a 3113-level archive, 21 arrivals have that form (`client-internals/decomp-findings-2026-10-25-f/02-link-arrival-expressions.md`).
- Link arrival text in the binary writer is the text line `dest x y w h newx newy`.
- Link destinations may be a file name or a URL. A destination that is neither `.graal` nor `.nw` is accepted as typed. **[prev]**

9. Signs

- **Edit signs** (key S): a list with X and Y fields, New, Delete. A sign at (0, 0) is not drawn on the board. Such a sign is used by index: a chest shows sign *n* when opened, and the GS1 `say n` command shows sign *n*. **[prev]**
- The special-codes list (`TShowCodes`) was merged into the sign dialog (change log v2.1); the class is gone from the 2.1 test builds on. **[log]**
- Text is stored in the sign alphabet with `#K(n)` escapes (see `formats/sign-text.md`). Empty trailing lines are removed automatically (1.41). **[log]** The encoder in the binary writer follows the alphabet in `formats/sign-text.md`.

10. Chests

- Dialog `TChestEdit`: item (index), a sign index, Sign button, Delete, Close. **[2.22]**
- Item list, in index order (unchanged from 2.05, checked against the 2.22 string table): `greenrupee, bluerupee, redrupee, bombs, darts, heart, glove1, bow, bomb, shield, sword, fullheart, superbomb, battleaxe, goldensword, mirrorshield, glove2, lizardshield, lizardsword, goldrupee, fireball, fireblast, nukeshot, joltbomb, spinattack`. A new chest defaults to item 1 and sign -1. **[2.22][prev]**
- The **Sign button** opens the sign editor for the chest's sign index. **[2.22]**

11. Baddies

- Dialog `TEditBaddy`: type selector, three verse fields (attack, when hurt, when it hurts the player), Delete, Close. **[2.22]**
- Ten types: Gray Soldier, Blue Soldier, Red Soldier, Shooting Soldier, Swamp Soldier, Frog, Spider, Golden Warrior, Lizardon, Dragon. **[prev]**
- Choosing type 4, 5 or 6 (Swamp Soldier, Frog, Spider) disables the verse fields. The writer saves empty verses for those types. **[2.22]**
- A new baddy starts with "Stop!" and "Ouch!". **[prev]**
- Loaders in builds up to 2.04 clamped baddy and chest positions; see `version-deltas.md`.

12. NPCs

- One dialog, `TEditNPC`: image, script, Test (syntax check), Style, Find, and the usual edit menu. **[2.22]**
- The script editor understands the editor markers `//#EDIT name value` and `//#COMMENTS text` used by predefined NPCs. **[log]** Predefined NPCs are `defnpc*.txt` files; when one is dropped on the level every occurrence of the named variable is replaced by the entered value. **[log]**
- *Character NPC* (`TEditCharacter`): a form that generates a walking-character script. Fields as in the 2.05 note. **[prev]**
- Edit flags (`TEditFlags`): the offline flag list; it is not saved in the level. **[prev]**
- NPC x and y are fractional tiles; the binary format stores them as two bytes (+32), so only whole and half tiles survive a binary save. **[inf]**

13. Keys

Delphi shift-state bits: 1 = Shift, 2 = Alt, 4 = Ctrl. **[2.22]**

Key	Action
Ctrl+N	New
Ctrl+O	Open
Ctrl+S	Save (save-all, see section 4)
Ctrl+Z, Alt+Backspace	Undo board copy
Ctrl+X, Shift+Del	Cut
Ctrl+C, Ctrl+Ins	Copy
Ctrl+V, Shift+Ins	Paste
Del	Delete the selection
L	Edit links
S	Edit signs
P	Play
M	Generate map image

Key	Action
Ctrl+Q	Batch re-save for gmaps: for each level name in the loaded map's level list the editor reads the file and writes it again with the writer for its extension. **[struct]** Seen in 2.17.1 and 2.22. Not found in the 2.05 export (the handler may simply be missing there).
F10	Toggle window border style
Esc	Warning box before leaving the editor

L, S, P and M are ignored while a combo box in the side panel has focus. Ctrl+Enter and a few Ctrl+letter chords are game-mode shortcuts handled by the same form.

14. Tabs, maps and multi-level editing

- Each tab keeps its own scroll position, selection, floating paste, undo stack and object lists. New tabs are titled `(new)`. **[prev]**
- Multi-level editing (v2.12): when an NPC calls `setmap`, the editor shows the neighbouring levels together. Drawing follows NPC y order. You scroll by dragging the player or by arrow keys; on maps a big cursor replaces the player (2.13). **[log]**
- Editing a level on a map marks near levels modified when you delete their NPCs; undo works across a level change (v2.16 fixes). **[log]**
- `TGraalMap` holds the gmap model and the generated-map seeds; `TEditorLevelCache` holds recent levels. **[prev]**

15. Other dialogs

`TLoadLevel` (open by name, with a browse button that clears the file dialog's options before running it), `TLogFrame` (log window, present since 2.17.1), `TShowImg` (present since 2.17.1; role not traced), `TEditVariable`, and the tile-array window. **[2.22]**

16. Open

- The mouse state machine and the exact drag/drop rules.
- Meaning of the two session-mode flags that gate chests and baddies.
- Whether the Ctrl+Q batch re-save converts formats or only normalises files (it reads with the magic-based loader and writes by extension, so a `.graal` file in any readable version comes out as `GR-V1.03`).
- Where "Unsupported file type!" is raised in 2.22. The string is in the 2.22 and 2.31 string tables, but no caller appears in the exported code.
- Three handlers that the second decompiler pass could not recover (no function found).
- How the type and offset fields of tile definitions are used.
- Whether arrival offsets are evaluated at warp time.

`tools/level-editor/formats-crossref.md`

How the editor relates to the format notes

Date: 2026-10-25. Cross-reference between the editor's loaders and writers (2.22, with deltas) and the existing format notes. Tags as in `README.md`.

`formats/level-binary.md` and `level-binary-client.md`

Topic	Editor behaviour (2.22)	Relation
Code width	13-bit for `GR-V1.02`/`1.03`, 12-bit for `GR-V1.00`/`1.01` and `Z3-V1.*`	Confirms the rule. Same in 1.39, 2.00, 2.05, 2.17.1.
Board run-length	The writer emits single runs and double runs (`ABAB`…, up to 64 pairs)	Confirms that stock writers emit double runs.
Section order	links, baddies, NPCs, chests, signs, each with its terminator	Confirms. The 2.22 writer appends a newline after each chest record.
Baddy section end	Ends at a record shorter than three bytes or a first byte ≥ 0x80. There is no x = 0 test.	Matches batch f; **`client-internals/level-editor-205.md` said otherwise and is corrected in this PR.**
Baddy record	Raw bytes (x, y, type, **not +32**), then `verse1\verse2\verse3`. Types 4–6 write empty verses (`\\`).	Confirms.
NPC record	`x+32, y+32, image#script` + newline	Confirms. NPC x, y in the text format are floats; only whole and half tiles survive the binary save.
Chest record	Four bytes `x+32, y+32, item+32, sign+32`	Confirms; sign index −1 is stored as byte 31.
`GR-V1.04`	Cache writer writes it when heights exist; the editor open path does not accept it	Matches `level-binary.md` (GR-V1.04 = V1.03 plus optional 9 × 9 `HEIGHTS`, 2.31).
Save format	`.nw` → text, anything else → `GR-V1.03`	New. No editor writes `GR-V1.00`–`1.02`, `Z3-V1.*` or `GR-V1.04`.

`formats/glevnw01.txt`

The text reader's rules are in `222-reference.md` section 4. New items for that file's notes:

- The `BOARD` layer field is ignored by the 2.22 reader. 2.05 also ignores it.
- `BOARD` rows are placed by (x, y, w); out-of-range or short rows are skipped silently, invalid base64 characters end a row.
- `HEIGHTS` is a 9 × 9 grid: up to nine lines of up to nine comma-separated decimals.
- `LINK` needs five tokens; a missing or empty new x or new y becomes `playerx` / `playery`.
- Tile ids from text are 12-bit (two base64 characters).

Batch f (`decomp-findings-2026-10-25-f`, PR #39)

Batch f finding	What the 2.22 editor adds
Link arrival fields hold numbers, `playerx`/`playery`, and `playerx`/`playery` plus a signed offset	The editor stores arrival text verbatim (no parsing); its own *warp field* tool writes `playerx` / `playery` by default and the literal numbers `0` or `61` at the level edges (rules in `222-reference.md` section 6). Offsets such as `playery+10` are hand-typed text. Whether the engine evaluates them was not traced (open point from batch f stays open).
Baddy section ends only at a first byte ≥ 0x80; x = 0 is valid	Matches the code in five builds (1.39, 2.00, 2.05, 2.17.1, 2.22): the test is on the first byte's sign, with a length check; no x test.
Sign alphabet and `#K(n)` escapes	The editor's sign dialog edits plain text; the binary writer maps it through the sign alphabet. The special-codes window of old builds listed the escapes (`#h`, `#A`, …).
Sign at (0, 0) and chest sign index −1	Same in the editor: a (0, 0) sign is not drawn, and a new chest has sign −1.
No file carries a `HEIGHTS` block	The text writer emits `HEIGHTS` only when the height overlay is marked modified; the binary writer never emits it. A level never touched by height tools has none.

GR-V1 notes (`formats/grv1-decoder`, `formats/seihai-boards.md`)

- A Seihai reader/writer for the editor must write by extension: `.nw` text, any other name `GR-V1.03`.
- The board's tile id range is 0–4095; the 12-bit text format cannot carry ids ≥ 4096, and binary levels from player worlds use ids ≥ 2048 in 64 % of boards (batch f). The tile → sheet mapping in `222-reference.md` section 5 covers the full range.

Correction made in this PR

`client-internals/client-assets.md` gave the tile mapping as `col = t % 128`, `row = t // 128`. The correct mapping is `col = (t % 16) + (t / 512) * 16`, `row = (t / 16) % 32`, the same as Seihai's renderer already uses.

`tools/level-editor/README.md`

Classic Graal level editor: notes by version

Date: 2026-10-25. Derived facts only: class and form inventories, string tables, file-format behaviour, and the behaviour of the editor code paths. No code, addresses or symbol names.

Reading order

1. `222-reference.md`: the primary reference. The editor built into the 2.22 client, described feature by feature.
2. `version-deltas.md`: every other build as a delta from 2.22 (1st Graal through 2.31, the New World betas, and the standalone GraalEditor).
3. `formats-crossref.md`: how the editor's loaders and writers relate to `formats/level-binary.md`, the GR-V1 notes, and the 2026-10-25 batch f findings.
4. `seihai-tracking.md`: the 2.22 feature list in implementation order, then per-version deltas, written as a checklist for a future version-selectable Seihai level editor. It is a tracking note only; nothing here implies an implementation.

Where the material was

- The built-in editor is the `TAdventure` main form in a second, editor mode of the client. It is present in the 2.22 client and in every earlier client the notes cover; the 2.31 client has lost the editor forms.
- Primary source: the 2.22 client's built-in editor, read from a decompile plus a second decompiler pass over handlers the first export had left out.
- Version differences: the 2.17.1, 2.31 and 2.05 rev 2 clients (decompiles), the 1.39 and 2.00–2.04 clients (decompiles and string/class inventories), 1st Graal and every build in the drive-clients inventory (string and class inventories only), and the two New World beta clients.
- Shipped documents in the 2.22 install tree, which are not decompiles and are the best evidence for *when* a feature appeared: the `newfeatures` change logs (1.41, 2.12, 2.17, 2.21, 2.22, 2.31), the stamp sets, the tile-definition files, the editor cursor GANI, and the editor-NPC example scripts.
- The standalone editor: `GraalEditor.exe` 2.3.1.1 is a Thinstall package (see `tools/graaleditor.md`). It was only checked for packing. It was **not** unpacked in this pass. Earlier notes already describe an unpacked image, and those are used as they stand. `Greaditor.exe` (2001, New World beta folders) is UPX-packed and looks like a separate editor; not unpacked either.

Tags

- **[2.22]** read from the 2.22 code or strings this pass.
- **[prev]** carried from an existing note (`client-internals/level-editor-205.md`, `tools/graaleditor-vs-205.md`, `tools/graaleditor.md`) and not re-read in 2.22. The dialog or class exists in 2.22; its contents are assumed unchanged.
- **[log]** from a shipped change log. The change log says it, the code was not traced.
- **[struct]** structure only. The decompile of that routine is dominated by obfuscated accessors or is missing, so only its shape (what it calls, what it tests) is known.
- **[inf]** inferred.

Decompiles are lossy

Every statement tagged [2.22] comes from decompiled Delphi code. The decompiles lose type information, merge branches, and show the program's own obfuscated field accessors as arithmetic noise. Where a handler could not be read cleanly it is tagged [struct]. Handlers that the first export did not contain were recovered with a second decompiler pass; the three that could not be recovered are listed in `222-reference.md` under *Open*.

What is new compared with the existing notes

The 2.05 note and the GraalEditor notes already cover the dialogs, the palette, undo, the GOBJSET01 sets, and the GR-V1.03 writer. This folder adds:

- 2.22 as the reference, with a per-feature table that says which parts were re-read in 2.22.
- A **correction** to `client-internals/client-assets.md`: the tile id → sheet mapping was wrong (see `222-reference.md`).
- A **correction** to the baddy section ending in `client-internals/level-editor-205.md` (x = 0 does not end the section).
- The 9×9 `HEIGHTS` block in the editor's text format, and where the editor writes it.
- The `BOARD` row rules of the text reader, including that the layer field is not read.
- Defaults and edge rules of *Turn selection into a warp field*, including the `playerx` / `playery` arrival defaults and the edge cases that write `0` and `61`.
- Save format chosen by the file extension, and the fact that none of the editors checked (1.39, 2.05, 2.22) writes `Z3-V1`, `GR-V1.00`–`GR-V1.02` or `GR-V1.04` levels.
- `Ctrl+Q` (batch re-save) and `F10` (border toggle) in the key map.
- Flood fill with the right mouse button, multi-level editing on maps, editor control-NPCs, predefined NPCs, and the other change-log dated features.
- A class and format timeline across every build in the inventory, including that the New World betas have no chest or baddy editor.
- Chest position clamp (0–62) in loaders up to 2.04, removed in 2.05.
- One line on the standalone editors' packing.
- A Seihai tracking checklist.

Not changed

- No existing note was removed. Three were corrected in place (see above and the README entry).
- `formats/level-binary.md` and the batch f findings are cited, not edited.

Caveats

- The mouse state machine (drag, drop, select, double-click dispatch) is **structure-only**.
- The two obfuscated session-mode flags that gate chest and baddy load/save are **structure-only**; their meaning is only inferred.
- The standalone GraalEditor was not unpacked. Its behaviour comes from earlier notes.
- Class inventories use exact class-name strings in each exe. A class missing from the table may only have been renamed.
- Whether the stock client evaluates arrival offsets such as `playery+10` at warp time was **not traced**. The editor only stores the text.

`tools/level-editor/seihai-tracking.md`

Seihai level editor: tracking note

Date: 2026-10-25. Written for the Graal_analysis repository as a hand-off checklist. **It is not an implementation plan, and the Seihai client remains the main priority.** It can be copied into Seihai's `documentation/` as it stands.

Design intent (from the owner)

A future Seihai level editor is **version-selectable and profile-driven**, like the client and its version chain. The **2.22 editor comes first**; other versions are deltas applied as profiles. No implementation is started.

Suggested profile fields (names are placeholders): `formats_read`, `formats_write_by_extension`, `has_chests`, `has_baddies`, `has_heights`, `has_predefined_npcs`, `has_level_cache`, `tiledef_support`, `open_filters`, `load_clamps`, `dialogs`.

Part 1. The 2.22 feature list, in implementation order

Each item names the section of `222-reference.md` that holds the rules.

1. **Level model and the tile sheet.** 64 × 64 board of 16-bit ids, links, signs, chests, NPCs, baddies. Tile id → sheet: `col = (id % 16) + (id / 512) * 16`, `row = (id / 16) % 32`. (§3, §5)
2. **Read and write GLEVNW01 text.** `BOARD x y w layer data` with the exact skip rules, `LINK`, `SIGN`, `CHEST`, `NPC`, `BADDY`, `HEIGHTS` (9 × 9). Round-trip test against real levels. (§4)
3. **Read `GR-V1.00`–`1.03` and `Z3-V1.03/1.04`; write `GR-V1.03`.** Pick by extension. Reuse Seihai's existing binary reader. (§4; `formats-crossref.md`)
4. **Board view.** Scroll, 16-px tiles, tile overlays (blocking, foreground, terrain) from the type table, status bar. (§2, §5)
5. **Selection, copy, cut, paste, delete, floating paste.** (§6)
6. **Undo of board writes**, per tab, 128 records, rectangle snapshots. (§6)
7. **Tile palette.** Single tile, block pick, background tile, draw mode, flood fill with the right button. (§5, §6)
8. **Stamps.** Load `GOBJSET01` sets; place; add selection to the set; save the set. (§7)
9. **Link tools.** Link list and dialog; *turn selection into a warp field* with the `playerx`/`playery` defaults and the edge rules; open the destination in a new tab. (§6, §8)
10. **Sign tools.** List, x/y, text in the sign alphabet; (0, 0) signs. (§9)
11. **Chest tool.** Item list (25 items), sign index, Sign button. (§10)
12. **Baddy tool.** Ten types, three verses, types 4–6 without verses. (§11)

13. **NPC tool.** Image, script text with Test and Find; clone with the right button; predefined NPCs (`defnpc*.txt`, `#EDIT`, `#COMMENTS`). (§12)
14. **Tabs, Save (save-all), Save as, Close with prompt, New.** Write to a temporary file and rename, **not** delete-then-write like 2.22. (§4, §14)
15. **Keys.** The table in §13, including `F10`; `Ctrl+Q` batch re-save is optional.
16. **Maps.** Multi-level view, level cache, gmap map generation (`M`). (§14)
17. **Tile definitions.** `tiledefs` lines, type 0 and 1; type 4 after the semantics are traced. (§5)
18. **Play.** Needs the client. Out of scope for the editor itself.
19. **Editor control-NPCs.** `editnpc*.txt` with mouse variables. (§2) Late, depends on the script engine.

Part 2. Per-version deltas (as profile switches)

Profile	Switch changes relative to 2.22
2.17.1	none found
2.05 (rev 1/2)	no heights; no level cache; no tiles-array/log windows; no predefined NPCs; open filter `*.graal` only; person image and script are separate dialogs; text save has no `HEIGHTS`
2.04	as 2.05 and **no text format at all** (read or write); chest load clamp 0–62, NPC 0–63; object sets and tile defs present
2.00 – 2.0 rev3	as 2.04 without object sets and tile defs (object sets arrive in 2.04); rev 0–2 also open `*.zelda`
1.34 – 1.41	as 2.00; the baddy class is named baddy; draw mode and map key arrive in 1.41
1.28b2 – 1.33	baddy class named opponent
2nd – 1.1	no level class; character dialog absent before 1.28b2; flags dialog from 3rd Graal; 1st Graal has no chests or NPC dialog
New World betas	**no chests, no baddies**; heights and map present in beta 1; text format described by the beta's own format file
2.1 test builds	merged NPC dialog; predefined NPCs; no sign-codes window
2.31 / GraalEditor 2.3.1.1	editor is a separate program; writes text with `HEIGHTS`; the board section is claimed to be decimal in an older note (unchecked)

Open items to settle before any build

- Mouse state machine (drag, drop, double-click by kind): structure only.
- The meaning of the two chest/baddy mode flags.
- Tile definition `type` 4 and its x/y values.
- Whether arrival offsets are evaluated at warp time.
- `GraalEditor.exe` board format (decimal vs base64) against a real file.

`tools/level-editor/version-deltas.md`

Level editor: every other build as a delta from 2.22

Date: 2026-10-25. Base: `222-reference.md`. Tags as in `README.md`. "Present" below means the exact class name is in the exe's class table or string table; a class that looks absent may only have been renamed. Builds are named as in `client-internals/drive-clients-inventory.md`.

1. The standalone editors (one line each)

- `GraalEditor.exe` 2.3.1.1 (2005): a **Thinstall package** (Thinstall signature as in `tools/graaleditor.md`; a single 17 KB code section in a file of about 1 MB). **Not unpacked in this pass.** Earlier notes (`tools/graaleditor.md`, `tools/graaleditor-vs-205.md`) already describe an unpacked image and are used as they stand.

- `Greaditor.exe` (2001, in the New World beta folders): **UPX-packed** (UPX0/UPX1, high entropy). It looks like a separate New World era editor and is **not unpacked**. `levelformat.txt` in the nwb1 folder documents the text format those betas used.

2. Class timeline of the built-in editor

x = present in the exe. Columns are the editor dialogs and model classes. Builds that have the same row are merged.

Builds	Baddy class	NPC class	Level class	Chest dlg	Flags dlg	Person dlg	Char dlg	New NPC dlg	Objset	Tiledef	Map	Heights	Cache	Tiles-array	Log
1st Graal	TEditorOpponent	none	none	.	.	.	.	.	.	.	.	.	.	.	.
2nd Graal	Opponent	—	none	x	.	x	.	.	.	.	.	.	.	.	.
3rd–15th Graal, 1.1, 1.1.9	Opponent	Person (most builds)	none	x	x	x	.	.	.	.	.	.	.	.	.
1.28b2, 1.31–1.33, 1.3b	Opponent	NPC	TEditorLevel	x	x	x	x	.	.	.	.	.	.	.	.
1.34–2.04, 1.41	**Baddy**	NPC	TEditorLevel	x	x	x	x	.	.	.	.	.	.	.	.
2.04	Baddy	NPC	TEditorLevel	x	x	x	x	.	x	x	.	.	.	.	.
2.05, 2.05 rev1/rev2	(TComputer)	NPC	TAdventureLevel	x	x	x	x	.	x	x	rev1+	.	.	.	.
New World beta 1	none	NPC	—	.	x	x	x	.	x	x	x	x	.	.	.
New World beta 2, 2.1 test 2/3	none / (TComputer)	NPC	—	beta 2: .; test: x	x	.	x	x	.	x	.	beta 2 x	.	.	.
2.17.1, 2.22	(TComputer)	NPC	TAdventureLevel	x	x	.	x	x	.	x	x	x	x	x	x
2.31	—	NPC (model only)	—	.	.	.	.	.	.	x	.	x	.	.	x

Notes on reading the table.

- 2.05 and later keep the `TEditorNPC` name. The baddy list uses `TComputer`, so the `TEditorBaddy` class is absent from the 2.05+ rows. The old level class `TEditorLevel` became `TAdventureLevel`, which the table does not track by name. **[inf]**
- `TEditPerson` / `TPersonAction` (image dialog plus script dialog) were merged into one `TEditNPC` in the 2.1 test builds. `TShowCodes` (the sign special-codes window) went at the same time: the 2.1 change log says the sign codes and add-NPC windows were merged into other windows. **[log]**
- The 2.1 test builds and the New World betas are **forks of the 2.0x line**; both carry the 2.05 text loader, but they differ in which dialogs are present. The New World betas have **no chest or baddy dialog and no chest or baddy class**. That matches the two obfuscated flags in 2.22 that switch chests and baddies off in the writers. **[inf]**
- `TObjectSet` (the *new set* dialog) is absent in 2.1 test, New World beta 2 and 2.17+, while the stamp sets still load in 2.22; the set-creation step moved elsewhere. **[inf]**

- 2.22 and 2.17.1 have the same class list. The 2.21 and 2.22 change logs contain no editor entries. No functional difference between the two editors was found; function shapes and section strings match, but a line-by-line comparison was not done.
- 2.31 has none of the dialogs. The level model classes (links, signs, chests, heights) and the GR-V1.04 reader stay because the game needs them, but the open, save and text readers are gone.

3. Formats, per build

Build	Reads	Editor save	Open filter	Notes
1st Graal	`Z3-V1` family only	not traced	`*.graal, *.zelda`	**structure-only** (inventory level).
2nd–1.39	`GR-V1.00`–`GR-V1.03`, `Z3-V1.03/1.04`	`GR-V1.03` always	`*.graal, *.zelda`	Chest position clamped 0–62 on load, NPC position 0–63 (GR-V1.00 path). Baddy section ends at length < 3 or first byte ≥ 0x80. **[1.39]**
2.00 – 2.0 rev2	same	same	`*.graal, *.zelda`	Same loaders as 1.39 (same clamps). **[2.00]**
2.0 rev3 – 2.04	same	same	`*.graal`	**`.zelda` dropped from the dialog** at 2.0 rev3. The reader still accepts `Z3-V1`. Clamps remain. **[inf] [2.04]**
2.05 – 2.05 rev2	same **plus `GLEVNW01`**	`.nw` → text, other → `GR-V1.03`	`*.graal` (text files should still open, since the loader reads the magic **[inf]**)	**GLEVNW01 first appears in 2.05.** No `HEIGHTS` in the editor path. The chest clamp is gone. **[2.05]**
NW beta 1	same	`.nw` / `.graal`	`*.nw, *.graal`	Text format documented in the beta's own `levelformat.txt`.
NW beta 2, 2.1 test 2/3	same	same	`*.graal, *.nw`	Dialog order switches to `*.graal` first.
2.17.1	same	same, plus `HEIGHTS` in text when the overlay is modified	`*.graal, *.nw`	`GR-V1.04` is accepted by the weblevels cache path only.
2.22	see `222-reference.md`		`*.graal, *.nw`	Reference.
2.31 client	`GR-V1.00`–`GR-V1.04`, `Z3-V1`; **no `GLEVNW01`**	none (no editor)	—	`GR-V1.04` = `GR-V1.03` plus an optional 9 × 9 `HEIGHTS` block. See `formats/level-binary-client.md`.

Build	Reads	Editor save	Open filter	Notes
GraalEditor 2.3.1.1	`*.graal, *.nw`	text with `HEIGHTS`; GR writer strings present		**Conflict.** `tools/graaleditor.md` says its `BOARD 0` section is 64 lines of decimal tile indices and not accepted by the client loader. The 2.22 built-in writer produces base64 rows. This was not checked against a GraalEditor-written file in this pass. **[prev]**

Verification notes.

- The clamp rows are from the loader code shape (a compare against 62.0 for chests and 63.0 for NPCs in the 1.39, 2.00 and 2.04 loaders; the compares are absent in 2.05, 2.17.1 and 2.22). A level with a chest at x = 63 therefore loads at 62 in ≤ 2.04. **[struct]**
- No editor writes `Z3-V1` or `GR-V1.00`–`GR-V1.02` (checked 1.39, 2.05 via the existing note, and 2.22).

4. Change-log dated features

From the shipped `newfeatures` files, dated by the heading each entry sits under. Not by the code; the 2.22 column says whether the 2.22 build is after that version (all are, except the 2.3 split).

Feature	Change-log version	In 2.22
Draw by dragging with a single tile chosen; M builds a map PNG; status bar shows mouse position and tile; double-click sets the background tile; fields outside the level not selectable; drag a text file to make an NPC; trailing empty sign lines removed; new level uses the chosen background tile	1.41 and 1.41 rev 1	yes
Flood fill with the right mouse button (Shift: complete objects only; nothing selected: background tile)	2.00	yes
Object sets in the tile-objects folder, selectable in the editor; `addtiledef`, `addtiledef2`, `removetiledefs`	2.04	yes
Predefined NPCs (`defnpc*.txt`, `#COMMENTS`, `#EDIT`); sign-code and add-NPC windows merged away; multi-tile selection in the tile panel; F4 on a tutorial level opens a new empty level	2.1	yes
Multi-level editing when an NPC called `setmap`; scroll by dragging the player or by arrow keys	2.12	yes
Single levels get scrollbars again; big cursor on maps; Play saves all modified levels; editor control-NPCs (`editnpc*.txt`, mouse variables); M fixed	2.13	yes
Dragging baddies, NPCs and chests onto a level can be released and then moved; level caching and player position on Play fixed	2.13 rev 3	yes

Feature	Change-log version	In 2.22
Mouse wheel scrolls; arrow-key scrolling on single levels	2.15	yes
Button showing the selected tile rectangle as text; `showimg` and `putnpc` allowed in editor NPCs; flood-fill, undo and NPC-delete fixes on map levels; M faster	2.16	yes
M works on gmaps	2.16 rev 7	yes
Editor and online client become separate exes; the editor writes a copy and renames it so a crash no longer deletes the level; filename cache	**2.3**	**no**

5. Deltas as a checklist (per build, relative to 2.22)

- **2.17.1**: none found.
- **2.05 (all revisions)**: no heights, no level cache, no tiles-array and log windows, no predefined NPCs, separate person-image and script dialogs, `TShowCodes` window for sign codes, open filter `*.graal` only. Save and load otherwise identical to 2.22. New World style map parser present from rev1 but dormant.
- **2.04 – 2.0 rev3**: no `GLEVNW01` reader or writer at all; object sets and tile definitions present from 2.04. Chest position clamp 0–62 on load.
- **2.0 – 2.0 rev2 and 1.3x**: open filter includes `.zelda`; the `Z3-V1` reader is live. Same clamps.
- **1.28b2 – 1.33**: baddy class still named opponent. Character NPC dialog present.
- **Before 1.28b2**: no character dialog, no level class; 2nd Graal adds chests and person dialogs; 3rd Graal adds the flags dialog; 1st Graal has no chest or NPC dialog.
- **New World betas**: no chest and no baddy; heights and map present in beta 1.
- **2.1 test builds**: combined NPC dialog, predefined NPCs, tiledef type, no sign-codes window.
- **2.31 and later client**: no editor; the standalone editor takes over (Thinstall). The 2.31 start screen has an *Editor* button that launches `GraalEditor.exe` (seen live, `client-internals/live-checks-editor-offline-sound.md`).

`tools/level-tools-gaps/README.md`

Level-tool gaps

Date: 2026-11-01

What the 2.22 generator reference and the level converter leave open, filled only where a note or a static read already states the layout. The original programs were not run. No image from the generators folder is committed.

`tools/generators-222/` and `tools/level-convert/` are on `main`. This folder does not replace them.

Z3-V1.02 and GR-V1.04

`formats/grv1-decoder/grv1.py` reads both.

- `Z3-V1.02` is 11-bit codes. Only the 1998 Java Zelda Online games read it. The baddy record is the same 3 bytes plus a newline the `.03` reader already accepts. No real `.02` file is in the notes; the tests are synthetic. **[doc]** `formats/level-binary.md`, `client-internals/precursors-and-fan-builds.md`.

- `GR-V1.04` is `GR-V1.03` (13-bit board, NPCs, chests) plus an optional `HEIGHTS` block after the signs: an empty line, `HEIGHTS`, at most nine lines of at most nine comma-separated decimals, `HEIGHTSEND` (case-insensitive). The same block is kept on any magic. **[2.31]** `formats/level-binary-client.md`.

`tools/level-convert` now loads those two magics. Saving `.graal` still writes `GR-V1.03` and drops `HEIGHTS` with the existing warning. Saving `.zelda` still writes `Z3-V1.04`. `tools/file-formats/gwebl.py` uses the same 11-bit width for `Z3-V1.02`.

graal-txt and nw-txt

Both executables are in the generators folder on Drive. The March 2003 `levelgendoc.html` describes the nw-gmap build (dashed `.nw` names and a `.gmap`), not these two. `txtlevel.py` is the file-name and container contract from a static read:

Build	Level name	Body	Map file
graal-txt (2004-01-18)	`prefix` + column + `-` + row + `.graal`	`GR-V1.03`, because the name does not end in `.nw`	`bigmap.txt`
nw-txt (2001-06-30)	`prefix` + column + row + `.nw` (no dash)	`GLEVNW01`, because `.nw` selects the text writer	`bigmap.txt`

Column letters are a plus the index mod 26. A leading letter is added only when the map is at least 26 levels wide. The row is 1-based. It gains one leading zero when the map is at least 10 levels tall, and another when it is at least 100. `bigmap.txt` is one row per line of `"name","name"`. **[bin]**

The reference writes those names, an empty-object level body, and the bigmap. It does not paint the picture. Links are the terminator only.

DungeonMaker 0.9

`docu_dungeongenerator.txt` names the styles (maze, onion, spiral, rogue, snake, square spiral), the symmetry values 0–4, `DOUBLE`, and the onion and spiral option bits. It shows pictures of the results. It does not say how the crawler walks, where it turns, or how a room is stamped. The `layouts` folder holds `rooms0`–`rooms9`, `roomsX`, matching `stats*` files and `constants`. The manual does not describe those bytes.

There is no crawler reference here. `tools/generators-222/dungeon.py` already draws the shapes the manual names, and it says those shapes are not the library. **[doc]**

Water, mountain and sand edges

The March 2003 manual names water, mountains and sand as colours. It does not name edge tiles. A static read of graal-txt shows colour 2 (water) and colour 3 (mountains) each mark a 6-pixel cell from its four corners, and a later pass blits small stamps from neighbour flags. Colour 5 (sand) is tested in the per-pixel pass. The colour match is the sum of absolute channel differences, strictly under 32, the same limit as the nw-gmap reference.

The stamp-to-shore table is a long branch list. Publishing a guess would invent a layout, so `generators-222/levelgen.py` still paints one fill tile for those colours, and this folder does not add another painter. **[doc][bin]**

gif2mng

`gif2mng.py` writes the subset `../gif2mng.md` states, on synthetic frames:

- one frame → PNG
- two or more frames → MNG
- `-b` forces transparent index 2 and does not add a palette entry

The seven body-slot RGB values are not in the note (the draw-time roles are in `client-internals/body-colours-and-attributes.md`, which is not a file palette). `-b` therefore leaves the RGB entries as they were. TERM is a chunk name in the program; its delay fields are not specified, so it is not written. MHDR ticks and play time are 0. One FRAM of mode 1 precedes the frames. **[doc][bin]**

`tools/navigator/README.md`

Notes navigator

A local map of the markdown notes in this repository. It is a navigator, not a research note. Nothing is written back into the repo.

Start

From the repository root:

```
python3 tools/navigator/navigator.py
```

Then open the page in a browser on this machine. The process listens on the loopback address, port 8765. That port is not a lab server port.

The bind address defaults to the loopback address, built the same way as `tools/lab-servers/listserver.py`. Any other host is refused. `--port` and `--repo` override the port and the checkout.

Local only

The process does not phone home: no telemetry, no analytics, and no outbound request. It serves only markdown files inside the repository, and it refuses a path that leaves the repository. Raw HTML in a note is escaped, not executed. Search compares the title and the path on this machine. There is no search service.

Home map

The home page is one screen with three parts:

- An architecture diagram of the corpus areas: client versions, protocol, formats, NPC scripts, Graal3D, Torque, tools, labs and archives. Choosing an area opens that cluster.
- A web of the relative markdown links inside the focused area. Choosing a node opens the note. A large area shows the linked notes, not every file. Search filters that web; it is not the whole page.
- A version map from left to right: 1998 and 1.x, 2.x, 3.x, 4.x, 5.x, 6.x, and Graal3D. A node is linked only when that note is in the repository.

The note page is the reader, with a strip of the notes that link here and the notes this page links to.

Tests

```
python3 -m pytest tools/navigator -q
```

`tools/npc-control.md`

NPC Control (RC 1.41, 2002)

Date: 2026-10-21

Source: static read of the Delphi `rc141.exe` ("GServer2 Remote Control 2002/05/02"), the first RemoteControl build that contains NPC Control. Also `NPCControl1.3` (a 2001 docs bundle with no binary). Tags: **[bin]** read from the binary · **[doc]** from the 2001 text files · **[inf]** inferred.

The later 2004 RemoteControl is in `remotecontrol.md`. This note covers the 2002 build, which is where NPC Control first appears. **Protocol layouts for `seihai-rc`:** `../protocol/npc-control.md`.

What the NPCControl 1.3 bundle is [doc]

It has no executable. It holds:

- `npccontrol.txt`: NPC Control connects to port 14950 on the NewWorld game server. Every NewWorld account could connect there; on the main server only accounts with admin level 4 or higher could.
- `npcserverdoc.txt`: the 2001 NPC server document. It is byte-identical to the separately uploaded "NPC Server Document Graal2001" (notes in `npcserver-2001.md`).
- `waytracker.txt` / `waytracker-show.txt`: two small NPCs. One records the player's walk as `dir - length` pairs into a string (for `this.ways` arrays of police NPCs), and the other displays it.
- `graal2001classes/`: ten class scripts that NPCs `join`: bomys, bomypolice, police, horses, gralats, minerocks, housecontrols3, castlecontrols, ctfcontrols, ctfflags.

The bundle says "Adding/Viewing/Changing classes doesn't work yet", so at 1.3 class editing was still missing from the tool.

Program shape [bin]

RC 1.41 is RC and NC in one window. NPC Control adds these forms to the older RC: `NPCList`, `AddNPCFrame`, `LocalNPCs`, `ClassesList`, `AddClass`, `WeaponsList`. The same release also adds `ServerList` (listserver login), `FileBrowser`, and `ShortMessage`.

Two `TGraalClient` sockets: one to the game server (RC) and one to the NPC server (NC). A third subclass, `TServerListClient`, talks to the listserver.

Framing (both sockets) [bin]

- Message: `GCHAR id` + payload + `\n`. Messages are batched and flushed when the buffer reaches 0x1000 bytes.
- Frame: `u16be length` + zlib body (Delphi zlib 1.1.2). There is no type byte, no cipher, and no per-message junk byte on either socket in this build.
- Large uploads: the client first sends id 50 (`PLI_RAWDATA`) with a GINT3 length, flushes, then sends one frame that holds `GCHAR id` + header + raw bytes with no newline.

- Receive: id 100 (`PLO_RAWDATA`) switches the reader into raw mode for the next *length* bytes.
- If more than 256 KB is queued unsent, the socket is dropped.

Logins [bin]

Socket	First message	Notes
RC	id **1**, `GSERV023`, `str account`, `str password`	no key byte
NC	id **3**, `NCL11012`, `str account`, `str password`	no key byte; sent right after the NC socket connects

The id is the client type that later gs2lib names: 1 = RC, 3 = NC. The 2004 RC moved to type 6 (`GSERV024`, gen 5); the NC login did not change between 2002 and 2004.

Straight after the RC login the client sends one `PLI_PLAYERPROPS` (2): - prop 33 (ADDITFLAGS) = 1, only when "Ignore mass messages" is ticked, - prop 0 (NICKNAME) = the nickname from the options, - prop 53 (PSTATUSMSG) = the selected status index.

Finding the NPC server [bin]

The RC never asks the user for an NPC-server address. It finds it like this:

1. Any player in the RC player list whose **nickname contains `(Server)`** is treated as an NPC server.
2. For each such player the RC sends id **94** (`PLI_NPCSERVERQUERY`): `GSHORT player id` + the text `location`. It does this when the player appears, and again whenever the NC socket drops.
3. The server answers with id **79** (`PLO_NPCSERVERADDR`): `GSHORT player id`, then `ip,port`. The RC acts on it only if the NC socket is not already open and only if that player's nickname contains `(Server)`.
4. The RC opens the NC socket to that address and sends the NC login above. The status bar shows "Connected to NPC-Server".

The defaults (the loopback address and `14804`) in the binary are unused leftovers of the old start dialog. **[inf]** The 2001 doc's "(NPC Admin)" PM interface (see `npcserver-2001.md`) is the older channel that this socket replaced.

Routing on the NC socket [bin]

Received ids 33, 80, 157–164, and 167 on the NC socket go to the NC handler. Everything else, including 16, goes to the normal RC handler.

NC → client (ids handled)

Id	gs2lib name	Layout as parsed	UI
16	DISCMESSAGE	rest = text	disconnect box
33	NPCWEAPONADD	`str name`, `str image`, `GSHORT len`, `len` bytes of script	weapon script editor
80	NC_LEVELLIST	rest = level list, one per line	level list editor
157	NC_NPCATTRIBUTES	rest = attribute text, one per line	attribute viewer
158	NC_NPCADD	`GINT3 id`, then props: `GCHAR 50` + `str name`, `GCHAR 51` + `str type`, `GCHAR 52` + `str level` (any order, any subset)	adds or updates a row in the NPC list
159	NC_NPCDELETE	`GINT3 id`	removes the row

Id	gs2lib name	Layout as parsed	UI
160	NC_NPCSCRIPT	`GINT3 id`, rest = script	script editor
161	NC_NPCFLAGS	`GINT3 id`, rest = flags, one per line	flag editor
162	NC_CLASSGET	`str name`, rest = script	class editor
163	NC_CLASSADD	rest = class name	adds to the class list
164	NC_LEVELDUMP	rest = text	dump viewer
167	NC_WEAPONLISTGET	repeated `str name`	weapon list

Id 188 (`NC_CLASSDELETE`) is not handled in 2002; it arrived with the 2004 build.

Client → NC (ids sent)

Id	gs2lib name	From	Payload
3	(login)	connect	see Logins
79	RC_CHAT	console line typed in NC mode	text
103	NC_NPCGET	NPC list: double-click, Enter, Attributes	`GINT3 id`
104	NC_NPCDELETE	NPC list: Delete (with confirm)	`GINT3 id`
105	NC_NPCRESET	NPC list: Reset	`GINT3 id`
106	NC_NPCSCRIPTGET	NPC list: Script	`GINT3 id`
107	NC_NPCWARP	NPC list: Quick warp, or the warp dialog	`GINT3 id`, `GCHAR x*2`, `GCHAR y*2`, level
108	NC_NPCFLAGSGET	NPC list: Edit flags	`GINT3 id`
109	NC_NPCSCRIPTSET	script editor: Save	`GINT3 id`, script
110	NC_NPCFLAGSSET	flag editor: Save	`GINT3 id`, flags
111	NC_NPCADD	Add NPC dialog	comma text of name, id, type, scripter, start level, x, y. Type is a drop-down: `OBJECT`, `ANIMAL`, `POLICE`, `HORSE`, `BOAT`, `HOUSEC`
112	NC_CLASSEDIT	class list: open	class name
113	NC_CLASSADD	class editor: Save	`str name`, rest = script
114	NC_LOCALNPCSGET	Local NPCs dialog	level name
115	NC_WEAPONLISTGET	console button	—
116	NC_WEAPONGET	weapon list: open	weapon name
117	NC_WEAPONADD	weapon editor: Save	`str name`, `str image`, rest = script
118	NC_WEAPONDELETE	weapon list: Delete	weapon name
150	NC_LEVELLISTGET	console button	—
151	NC_LEVELLISTSET	level list editor: Save	level list

Id 119 (`NC_CLASSDELETE`) is not sent; there is no class delete button yet.

All NC texts (scripts, flags, classes, weapons, level list, dumps) open in one shared editor window with a mode number. Save sends 109 (NPC script), 110 (NPC flags), 113 (class), 117 (weapon), or 151 (level list) according to that mode. Attributes (157) and dumps (164) are view-only and can be saved as `.rtf`.

The Add NPC dialog has the same fields as the 2001 PM command `add npc` (name, id, type, scripter, level, x, y), so id 111 is that command moved onto its own packet. **[inf]**

Local settings [bin]

- `controlconfig.txt` next to the exe: `nickname=`, `nomassmessages=`, `nohtmlinpms=`, `nohtmlimages=`, `attachaway=`.
- Registry `HKCU\Software\Graal`, section `RemoteControl`: values `accountname` (plain) and `password` (obfuscated, below).

Password obfuscation

The same scheme is used in every RC build from 1.33 to 1.41, and it also hides the listserver host name inside RC 1.41 and the 2.05 client.

1. Build a 24-byte block: `len` (one byte), the password (at most 23 bytes), random padding up to 24 bytes.
2. Encrypt it as three 8-byte blocks with DES in ECB mode. The cipher is a d3des port whose key bit order is LSB-first, so it equals standard DES with each key byte bit-reversed.
3. Write each output byte as two characters, high nibble first, each nibble `+ 0x20`. The result is 48 printable characters.

The RC password key is the 8-byte string `\!x<|.ak`. Decoding is the reverse; a block whose length byte is 24 or more is rejected.

Listserver login (new in 1.41) [bin]

- Host: stored as a 48-character ciphertext (scheme above, key bytes `2B 77 77 A7 B4 31 21 41`). It decodes to `listserver.graalonline.`, to which the code appends `com`. Port `14922`.
- Messages: id 0 `rc` (version string), then id 1 `str account`, `str password`.
- Replies handled: 0 (server list: `GCHAR count`, then per server `GCHAR field count` + that many `str` fields), 1 (one more server entry), 2 (end, disconnect), 4 (error text, disconnect).
- The server list shows name and player count. Connect uses the chosen server's ip and port.

The same key and nibble encoding hide the 2.05 client's list host, which the 2.05 notes left open: 2.05 rev 2 decodes to `<list-host-classic>`, port `14921`, version string `newmain`. **[bin]**

`tools/npcserver-2001.md`

Graal2001 NPC server: notes from the 2001 design document

Source: `NPC Server Document Graal2001.rar`, which holds one file, `npcserverdoc_Graal2001.txt` (13,520 bytes, sha256 `59edd6c1…57fd0dc`). It is byte-identical to `npcserverdoc.txt` in the NPCControl1.3 archive. Everything below is paraphrased; nothing here comes from a binary unless marked **[checked]**.

1. What the NPC server is

- A separate program that **logs into the GServer like an ordinary player** and owns the NPCs of a configured set of levels.
- Main loop, every **0.1 s**: read from GServer, run all NPC scripts, push changed NPC attributes to players, push changed player attributes to GServer and players, then sleep out the rest of the tick. So 0.1 s is the smallest useful `timeout` / `sleep` on the server side.
- Startup order: load options, connect to GServer, then (only if connected) load the level list, then load NPCs from its database.

- Data it receives from GServer: player attributes; temporary level state (bombs, explosions, bush removal, dropped items and the like); PMs; game time; levels to update or reload; `server.` flags being added or removed.
- Only some of what the NPC server computes is forwarded to clients; GServer decides.

Implied GServer link (inference, needs a capture to confirm)

Direction	Content named in the doc
GServer → NPC server	player props, level temp objects, PMs, time, level reload, server-flag add/del
NPC server → GServer	NPC props, player props (when scripts change a player), PM replies, weapon grants (`addweapon`/`toweapons`), warps (`setlevel2`)

The doc does not give packet ids or a port. From other threads: NPC Control 1.3 talks of port **14950** on the NewWorld host and admin level ≥ 4, and RC 1.41 logs into the NPC server with `NCL11012` on default the loopback address on **14804**. **[checked]** `RemoteControl.exe` contains `NCL11012`, `14804`, `Connected to NPC-Server`, and UI strings `Add NPC`, `Add Class`, `Classes`, `NPC List`, `NPC Level List`, `Script from Class`. These match the PM commands in section 5 one for one, which suggests RC 1.41 moved the 2001 PM interface onto its own connection. That last step is an inference.

2. Server-side scripting model

- Scripts run on the server by default. A line `//#CLIENTSIDE` splits the script: everything after it is shipped to clients. Clients only read NPC attributes and never propagate their own changes, so the client half is for visuals and sound.
- Server-only events: `created` (first run), `initialized` (reloaded, such as after an NPC-server restart), `playerenters`, `playerleaves` (not on disconnect), `playertouchsme` (needs `setshape`), `playerchats`, `npcwarped`, `exploded`.

3. Commands

Flow and core: `destroy`, `if`, `while`, `for`, `break`, `continue`, `sleep`, `set`, `unset`, `setstring`, `message`.

NPC appearance and state: `setgif`/`setimg`, `setgifpart`/`setimgpart`, `setshape`, `drawoverplayer`, `drawunderplayer`, `dontblock`, `blockagain`, `show`, `hide`, `noplayeronwall`, `showcharacter`, `setcharprop`, `setcharani`, `setchargender`, `putbomb`, `putexplosion`, `putexplosion2`, `setarray`.

Server-side additions: `canwarp`, `canwarp2`, `cannotwarp`, `callnpc`, `with (players[i])`, `with (getplayer(acct))`, `with (npcs[i])`, `with (getnpcs(name))`, `join`, `warpto`, `tokenize`, `tokenize2`.

Acting on the current player: `setplayerprop`, `setani`, `setgender`, `say`, `enableweapons`, `disableweapons`, `toweapons`, `addweapon`, `removeweapon`, `setlevel2`, `freezeplayer2`, `unfreezeplayer`.

Behaviour worth recording:

- `setshape 1,w,h`: the server never loads images, so it needs a rectangle to detect touches. Type 0 ("use the image") was planned, not built.
- `noplayeronwall`: `onwall`/`onwall2` stop counting players (the doc's example is the Elephant, which always has players on it).

- `canwarp` / `canwarp2`: database NPCs only. `canwarp` warps exactly as a player would (used with recorded waytracker paths for police NPCs). `canwarp2` is for wandering overworld NPCs: normal links, checks `onwall` at the destination, refuses small links such as house doors.
- `with`: temporarily swaps the current player or NPC. Commands inside are skipped if the target does not exist. Local variables still resolve to the caller, and the code still runs from the calling NPC's script. `sleep`, `break` and `continue` inside `with` leave the swapped context in place (a documented hazard).
- `join class`: loads `class.txt` from the scripts folder. Each NPC holds a list of script objects (its own first, then each joined class). A class already loaded by another NPC is shared, not copied, so updating a class updates every member. Later scripts can override functions.
- `warpto`: moves the NPC to another level; database NPCs only.
- `addweapon`: like `toweapons`, but takes the weapon script from the database (the weapon must already exist). Used by the Graal2001 trading house.
- String helpers: `strlen`, `startswith(part,str)`, `indexof(part,str)` (−1 if absent), `#e(start,len,str)`, `#T(str)` trim, `tokenscount`, `#t(i)`. `tokenize` splits on spaces and commas; `tokenize2` adds extra delimiters.

4. Variables and flags

Kind	Scope	Persisted
`name`	current script or class	no
`this.name`	current NPC	yes, for database NPCs
`level.name`	the NPC's level	no
`server.name` (number)	this NPC server only	no
object attributes	as in the client, plus `playerlastdead` (last death or login time; Graal2001 only)	—
flags	current player	yes, in the account
`client.` flags	current player, also sent to the client, and the client can change them	yes
`this.` flags	current NPC	yes, for database NPCs
`server.` flags	world-wide, shared with other NPC servers of the same world, never sent to players	yes

So `server.` *numbers* are local to one NPC server, while `server.` *flags* are shared across the world.

5. Admin interface (PM based)

Admins control the NPC server by sending PMs to its player login. Only senders whose nickname carries the `(NPC Admin)` tag are answered. An unknown command such as `help` returns the command list. Multi-line commands put their payload on the lines after the first.

Command	Payload and effect
`add npc <name>`	next lines: id, type (≤6 chars), scripter, level, x, y. Ids must be unique and ≥1000 for database NPCs (built-ins such as gralats sit below). The level and x,y become the reset position.
`reset npc <name>`	resets every attribute except the script and returns the NPC to its start (the name is case-sensitive)
`update script <name>`	the following lines replace the script. A running `sleep` is cut, so a `reset npc` is often needed afterwards.
`get script <name>`	returns an NPC's or a class's script
`update class <name>`	replaces a class script, saved to the scripts folder; members pick it up at once

Command	Payload and effect
`get npc list`	lists database NPCs (id ≥1000) with positions
`dump vars` / `dump vars <name>`	dumps the level's or an NPC's variables
`reload level list`	rereads the configured text file of levels this server handles

6. Message codes

- `#F`: the current *player's* level name. It can differ from `#L` (the NPC's level), for example in `playerleaves` or inside `with (getplayer(...))`.
- `#a(i)` and similar: an index suffix addresses the i-th player in the level.
- `#e`, `#T`, `#t`: see the string helpers above.

7. Cross-check against the v2.05 client [checked]

Mostly a plain string search of the client executables. The second bullet comes from the 2.05 script command table.

- `Graal_v2.05_rev2.exe` already contains most server-side names: `setshape`, `canwarp`, `canwarp2`, `callnpc`, `join`, `tokenize`, `tokenize2`, `tokenscount`, `setplayerprop`, `setgender`, `setchargender`, `toweapons`, `setlevel2`, `freezeplayer2`, `unfreezeplayer`, `putexplosion2`, `setarray`, `startswith`, `indexof`, `//#CLIENTSIDE`, and the events `playerenters`, `playertouchsme`, `playerchats`, `exploded`, `initialized`. The string `warpto` is also in the file, but it is not a script command there.
- How 2.05 handles them, from its script command table:
  - In the table but with no handler, so parsed and ignored: `canwarp`, `canwarp2`, `join`, `freezeplayer2`, `unfreezeplayer`.
  - In the table with a handler, so they run on the client: `setshape`, `callnpc`, `tokenize`, `tokenize2`, `setplayerprop`, `setgender`, `setchargender`, `toweapons`, `setlevel2`, `putexplosion2`, `setarray`.
  - Not script commands at all: `warpto`, `noplayeronwall`, `cannotwarp`, `addweapon`, `removeweapon`. So in 2.05 the first group does nothing whatever the server says; it does not depend on server packet 44 (which tells the client an NPC server is present). A server freezes a 2.05 player with server packets 154 and 155 instead of the script commands.
- Not found in any client build checked (2.05, 2.171, 2.22): `playerleaves`, `npcwarped`, `noplayeronwall`, `cannotwarp`, `addweapon`, `removeweapon`, `playerlastdead`, `level.`. These look server-only.
- `getplayer`/`getnpc` appear in 2.171 and 2.22 but not in 2.05.

Open questions

1. Packet ids and port for the NPC-server ↔ GServer link (needs a 2001-era GServer or NPC server binary, or a capture). The RC 1.41 (2002) side of NPC Control is now mapped in `tools/npc-control.md`. It shows the PM commands above became packet ids one for one: `add npc` is NC id 111 with the same seven fields (name, id, type, scripter, level, x, y), and type is one of OBJECT, ANIMAL, POLICE, HORSE, BOAT or HOUSEC. The RC finds the NPC server as the player whose nick contains `(Server)` rather than `(NPC Admin)`. There is still nothing on the GServer-side link itself.
2. Where the NPC database and the level-list file live, and their formats.

`tools/README.md`

Tools

Cleaned notes on the classic Graal tool chain. Derived from observed behavior and format output. No raw decompile material.

Tool	Role	Notes
GraalEditor	Level editor (2.3.1.1)	Separate exe from 2.3 onward. Writes decimal BOARD 0 grids plus LINK/SIGN/CHEST/NPC/BADDY.
Built-in level editor	Editor inside each client (2.22 is the reference)	`level-editor/`: 2.22 reference, per-version deltas, format cross-reference, Seihai tracking checklist.
GraalShop	GANI editor	Sprite mode (GANI0001) and movie mode (GANI0002).
TerrainGenerator	Height-field world	Plasma midpoint displacement → GRMAP001 + GLEVNW01 with HEIGHTS. The 2.22 reference is `generators-222/terrain.py`.
DungeonGenerator	Stamp dungeon	8×11 layout → GLEVNW01 boards and GRMAP001. The 2.22 reference is `generators-222/dungeon.py`.
GraalLevelGen	Image-to-levels	December 2000 contract in `classic-tools/graal-level-gen.md`. The 2.22 nw-gmap reference is `generators-222/levelgen.py`. graal-txt and nw-txt names and `bigmap.txt` are in `level-tools-gaps/`.
gif2mng	GIF converter	PNG for one frame, MNG for more. `-b` snaps to body palette. The writer for that subset is `level-tools-gaps/gif2mng.py`.
RemoteControl	Staff client	Classic text codec on two sockets. Predecessor of RC2.
RemoteControl 2000–2002	Staff client lineage	`GSERV014` → `GSERV023`, option packets, id changes. See `remotecontrol-lineage.md`.
NPC Control (RC 1.41)	NPC-server client	`NCL11012` login, NPC-server discovery, NC packet ids. Tool history in `npc-control.md`; wire layouts for `seihai-rc` in `../protocol/npc-control.md`.
Fan server tools	ListServer, serverlist, 1.39 scripts, Guildmaker	See `server-tools-2001.md`. Warns about a password stealer in the win32 ListServer.
Classic listserver ELF	Client and game-server directory	`classic-tools/serverlist3.md`. Frame and login lines in `classic-tools/listframe.py`.
gserverstat	HTML leaderboards	`classic-tools/gserverstat.md`. Account blob in `classic-tools/account13.py`.
Character Designer / GChar	Offline looks	`classic-tools/character-designer.md`. `CHR: 1.00` line in `classic-tools/chrfile.py`.
TextConvert	Graal to New World levels	`classic-tools/text-convert.md`. The full chain (zelda, graal, nw, gmap, future package) is `level-convert/`.
NPC server (2001)	Server-side NPC host	Logs into GServer as a player, 0.1 s tick, PM admin commands, `join` classes. See `npcserver-2001.md`.

See the individual files for the measured contracts. `graaleditor-vs-205.md` compares GraalEditor and GraalShop with the editor built into the 2.05 client. - `lab-setup.md` — the offline Wine + GServer + listserver-stub lab used for the [cap] captures. - `relay-bridge/` — `room_bridge.py` (logging proxy for the 1998 GServ relay that can rename join rooms), `AppletRun.java` (runs the Java Graal / Zelda Online applets in a frame),

`pcap_lines.py` (prints the relay lines in a capture). See `../client-internals/relay-room-bridge.md`. - `lab-servers/` — our lab listserver (version 2, with tests), `run-lab.sh` and the gs2lib patch our GServer build carries. See `../server-notes/lab-servers-2026-10-30.md` for what our servers are and how they compare with GServer-v2 beta4. - `navigator/` — local browser for the markdown notes in this repo. It listens on the loopback address (port 8765) and does not phone home. See `navigator/README.md`. - `ghidra/` — headless salvage scripts for recovered binaries (`analyze_tool.sh`, `ghidra_scripts/ExportSalvageArtifacts.java`). Counts and decompiler text stay in gitignored `local-out/`. The prose summaries of those tools are in seihai-servertools (`recovered-drive-tools/`, `recovered-hacks/`).

File-format helpers (`file-formats/`)

Pure-Python helpers written for the notes in `formats/` and `client-internals/`. No client code.

Script	Does
`graaldes.py`	The client's DES variant (bit-reversed key bytes) and its 48-character string cipher. Decrypts saved passwords. See `client-internals/saved-passwords.md`.
`gwebl.py`	Decrypts and parses `weblevels/*.code` (GWEBL001). Also a GR-V board reader and writer and a GR-V level parser. See `formats/gwebl001-cache.md`.
`extract_tutorials.py`	Pulls the 13 embedded tutorial levels out of a 2.0x exe (needs `pefile`). See `formats/level-binary-client.md`.

`tools/remotecontrol-lineage.md`

RemoteControl builds 2000–2002

Date: 2026-10-21

Source: static read of seven Delphi RemoteControl executables (archive names `rc133`, `rc139rev2`, `rc1025`, `rc62101`, `rc8212001`, `rc201`, `rc141`). The forms were read from their DFM resources and the packet ids from the send call sites. Tags: **[bin]** · **[inf]**.

The archive names do not sort by date. "rc141" is the newest of the set, not "rc201".

Builds

Archive	Title string	Login token	Forms	Notes
rc133	"GServer2 Remote Control (C) Stefan Knorr" (no date)	`GSERV014`	7	oldest; accounts, players, send, update levels
rc139rev2	2000/8/19	`GSERV019`	15	adds chat window, flag lists, toalls, SQL account list, options, warp
rc1025	2000/10/25	`GSERV020`	15	same forms as 2000/8/19
rc62101	2001/01/27	`GSERV021`	15	same forms
rc8212001	2001/08/20	`GSERV022`	18	adds rights, comments, local ban; server-option fields leave the start dialog
rc201	2002/01/04	`GSERV022`	18	same forms, new main window layout

Archive	Title string	Login token	Forms	Notes
rc141	2002/05/02	`GSERV023`	26	adds NPC Control, server list, file browser (see `npc-control.md`)

The 2004 build in the Graal Reborn folder (`GSERV024`, type 6) is covered in `remotecontrol.md`.

What stays the same [bin]

- Framing: `u16be length` + zlib body (zlib 1.1.2), messages `GCHAR id` + payload + `\n`. No cipher, no type byte, no key byte in any of these builds.
- Login: id **1**, then the token, `str account`, `str password`. The default account and password are `super` / `super`.
- Password storage: `controlconfig.txt` for options, `HKCU\Software\Graal\RemoteControl` for the account and the DES-obfuscated password (scheme in `npc-control.md`). The same key appears in every build.

Start dialog and the old option packets [bin]

Up to 2001/01/27 the start dialog asks for server name (default: the loopback address), port (`14804`), account, password, and four server timers, each with a "don't change" box:

Field	Default	Sent after login as	gs2lib name
Respawn time (bushes etc.) in sec	10	55	RC_RESPAWNSET
Horse lifetime in sec	10	56	RC_HORSELIFESET
Alignment increment time in sec	1200	57	RC_APINCREMENTSET
Baddies respawn time in sec	60	58	RC_BADDYRESPAWNSET

Each value goes out as a GINT4 (four 7-bit groups, high first, each `+0x20`), only when its box is unticked. From 2001/08/20 the timers are gone from the dialog and 55–58 are no longer sent. In 2002/05/02 the address fields are gone too; the server comes from the listserver.

From 2000/8/19 the RC also sends a `PLI_PLAYERPROPS` (2) right after login with its nickname. Closing the player list with "Go in away mode" sends the nickname with `- away` appended.

Client → server ids per build [bin]

Constant ids at the RC socket's send call sites:

Build	Ids
1.33	1, 51–64
2000/8/19, 2000/10/25	1, 2, 6, 28, 55–58, 61–64, 67–79, 82
2001/01/27	1, 2, 6, 28, 55–58, 61–64, 68, 70–74, 76–79, 82
2001/08/20	1, 2, 6, 28, 61–64, 68, 70–79, 82–88
2002/01/04	1, 2, 6, 28, 61–64, 68–79, 82–88
2002/05/02	1, 2, 6, 28, 51–54, 61–64, 68–79, 82–92, 94, 96–98, plus the NC ids

Ids 51–54 changed meaning [bin]

In 1.33 the account dialog uses 51–54 and 59/60:

Id	1.33 use	Later gs2lib name
51	account list: Edit (get account)	RC_SERVEROPTIONSGET
52, 53	edit account: Apply	RC_SERVEROPTIONSSET, RC_FOLDERCONFIGGET
54	account list: Delete	RC_FOLDERCONFIGSET
59	player list: Edit (get player props)	RC_PLAYERPROPSGET
60	edit player: Apply	RC_PLAYERPROPSSET

From 2000/8/19 accounts move to 70–78 (`RC_ACCOUNT*`, `RC_PLAYERPROPS*2/3`), and 51–54 are unused until 2002/05/02 brings them back as server options and folder config. A server that has to talk to 1.33 cannot use the gs2lib names for 51–54. **[inf]**

Feature timeline (from forms) [bin]

- 1.33: player list (nickname, account, level, head, id), disconnect, edit player, message, message to all, account list (add, delete, edit), update levels. The edit-account form already carries the full account: rights (Load only, Graalian Police, Trusted Admin, Administrator, No rights, Blocked), look, position, items, weapons, opened chests, NPC flags, MP, AP, spin attack.
- 2000/8/19: chat window (RTF), server flags, toalls, SQL account list with a filter dialog, message editor, options (ignore mass messages, no HTML in PMs, no images in PMs, away on close), warp player.
- 2001/08/20: edit rights, edit comments, local ban.
- 2002/05/02: NPC Control (NPC list, add NPC, local NPCs, classes, weapons, level list), listserver login and server list, file browser with drag and drop, short message popup.

`tools/remotecontrol.md`

RemoteControl

Delphi staff client, predecessor of the GTK `RC2_2.05`. One process, 25 auto-created forms. Combined RC + NC in one program ("new RC version with file browser, server list and RC+NC in one program", v2.13). The live console holds two `TGraalClient` sockets:

- RC / game-server socket.
- NPC-server (NC) socket, opened when the RC connects to the NPC server.

Revised 2026-10-21 from a static read of `RemoteControl.exe` (Graal Reborn 2.2.2.0 folder). Tags: **[bin]** read from the binary · **[srv]** gs2lib/GServer only.

Logins [bin]

Socket	First message	Mode during login	Mode after
RC	type **6**, GCHAR key (`rand & 0x7F`), `GSERV024`, `str account`, `str password`	fixed zlib, no cipher, no type byte	**gen 5** (type byte + partial XOR)
NC	type **3**, `NCL11012`, `str account`, `str password` (no key byte)	fixed zlib, no cipher, no type byte	unchanged for the whole session

- Right after the RC login the client sends a short `PLI_PLAYERPROPS` (2). Its prop list is not traced.
- Type 6 is what gs2lib calls "RC2". This Delphi RC already uses it, so "RC2 = type 6" in the server tables means the gen-5 RC login, not the GTK program. **[bin][srv]**

- `GSERV024` and `NCL11012` are plain string constants here, unlike the game clients' char-by-char tokens. Default NC address fields are the loopback address and `14804`. The live address arrives in `PLO_NPCSERVERADDR` (79), which the RC handles. **[bin]**
- The default account and password fields hold `super` / `super`. **[bin]**

Codec

Classic Graal text messages (`GCHAR id` + payload + `\n`, G-integers with `+0x20`), batched and flushed at 0x1000 bytes. Framing and cipher are the client-side gen 5 described in `protocol/client-versions.md`: in dynamic mode each flush picks raw/zlib/bz2 by size, writes a type byte, and XORs the first 12 or 4 dwords (one LCG step per 4-byte group present, a partial last group included) with the `0x04A80B38`-seeded cipher LCG (Borland multiplier, `+ key`). Send and receive keep separate iterators with the same key. The NC socket never leaves the fixed zlib mode, so NC traffic has no cipher and no type byte. **[bin]**

Correction: the earlier note "compression mode chosen once" is wrong for the RC socket. After login the mode is chosen per flush. **[bin]**

Ids

Server → client, RC socket: 13, 16, 30, 35, 37, 55–59, 61–70, 72–74, 76, 77, 79, 100, 103, 180. **[bin]** Server → client, NC socket: 16, 33, 80, 157–164, 167, 188. **[bin]**

Client → server with a constant id at the call site **[bin]** (a few more are sent with a computed id): - RC socket: 2, 6, 28, 51–54, 61–64, 68–76, 78, 79, 82, 84, 86, 88–92, 94, 96–98. - NC socket: 3, 79, 103–119, 150, 151.

Names for all of these are in `protocol/packet-ids.md`. The NC set lines up with gs2lib's `PLI_NC_*` (103–119) and `PLI_NC_LEVELLIST*` (150/151).

Player list

Packet 0x37 creates or updates a `TServerPlayer`. The body is a property stream walked by the property parser. Id is a 14-bit integer. Constructor defaults include hearts = 3, sword = `sword1.gif`, body = `body.png`, level = `onlinestartlocal.graal`.

File and account operations

- Folder change: 0x5a.
- Upload framed by 0x9b / 0x5d / 0x9c.
- Delete requires the rights string to contain `w`.
- Account list and property get/set use the matching PLI_RC_* ids from gs2lib.

Local settings live in `controlconfig.txt`. RC2 later wrote the same family to `control2config.txt`.

`tools/respawn-lab/README.md`

Respawn lab scripts

Used for `client-internals/board-respawn.md`. Everything runs in the Wine lab from `tools/lab-setup.md` (loopback-only namespace, Xvfb :9).

Script	Use
`mklevel.py out.nw`	Writes a grass level with bush, sign, pot, stone, black stone and tall grass in a row at y 26, and six bushes at y 33.
`boardscan.py <pid>`	Finds the 64×64 i16 boards in a running client by the y-26 row pattern. The board base is the hit minus `(26*64+20)*2`.
`boardwatch.py <pid> <secs> out.jsonl <base>...`	Logs every tile change with a timestamp (10 ms poll).
`pos.py <pid>`	Prints the 2.05 rev 2 player x, y.
`g2cli.py`	Minimal gen-2 (`GSERV014`) client: `u16be` + zlib frames, `chr(id+32)` lines.
`gs13_respawn.py out.jsonl N`	Two `g2cli` sessions against GServer13: shape tests, then N 2×2 changes, and logs every `PLO_BOARDMODIFY` both sessions get.
`arrays_dump.py arrays.dat`	Prints the cut, lift and push/pull tables.

The GServer13 account is a load-only `guest`/`guest` made in the server's own account editor. No real account is involved.

`tools/server-tools-2001.md`

Fan server tools (1999–2012)

Date: 2026-10-21

Notes on the small server-side tools uploaded with the NPC Control bundle. Tags: **[bin]** read from a binary · **[src]** read from source or script.

Win32 ListServer by "Rick" (2003) — contains a password stealer [bin]

`ListServer.exe` (UPX-packed Delphi, 2003-03-26) is a stand-in listserver for Graal clients. It reads `servers.ini` (`[Options]` `ShowMore`, `List`; one section per server with `Name`, `IP`, `Port`, `Description`, `Language`, `Version`, `Players`, `URL`, `Favorite`) and answers each client connection with the list. `Favorite=Yes` prefixes the name with `P` so the client files the server under Gold.

Do not run it. On first start it:

1. checks a marker value `Drag` under `HKCU\...\Internet Settings` and sets it to `WindowsUser@` so this runs only once;
2. reads every value under `HKCU\Software\Graal\Client` (where the Graal client keeps its saved login) and joins them as `name:<tab>value` lines;
3. Base64-encodes that text and opens `iexplore` on a third-party URL (stored Base64-encoded in the exe) with the data appended.

That sends the user's saved Graal account details to someone else's web server. The About box also opens a <graalonline-site> URL whose parameter is an insulting message. The tool's listserver behavior is not otherwise interesting.

gs2emu-serverlist (TheDgtl snapshot `de66a3f`) [src]

The C++ listserver used by Graal Reborn, an earlier form of the `graal-serverlist` already summarized in `protocol/listserver.md`. What this snapshot adds:

- `settings.ini`: game servers register on port 14900, clients connect on 14922, and pre-2.2 clients on 14923 (`clientPortOld`). Options for duplicate limits, max servers, and `auth`.
- MySQL tables: `graal_users` (account, password, salt, a second password/salt pair, activated, banned, profile fields), `graal_servers` (name, ip, port, player count, max players, description, url, language, two-character type, version, uptime, player list), `graal_servershq` (per-server HQ password, max/current level, parked flag), `graal_ipban`, `graal_guilds`, `graal_guilds_members` (account, guild, nickname, rank).
- Account passwords are `md5(md5(password) + salt)` with a three-character salt. Optional phpBB login and vBulletin friend lists.

GServer 1.39 web scripts (2001) [src]

- `gserverstats.pl` (TTH): a CGI page that reads the server's text outputs `playercount.txt`, `accountcount.txt`, `serveruptime.txt`, `toppkers.txt`, `topdeaths.txt`, `toponline.txt`, `topap.txt`. The top lists are `nickname:value` lines; online time is in seconds.
- `accountcreator.php` (The Expanse server): self-service sign-up into the GServer 1.39 MySQL `accounts` table. Columns used: `accname`, `encrpass`, `email`, `blocked`, `onlyload`, `adminlevel`, `adminworlds`, `lastused`. The script enforces Hotmail addresses and 20-character alphanumeric account names. Its welcome mail lists the 1.39 chat commands: `sethead`, `setsword`, `setshield`, `setnick`, `showname`, `showap`, `showdeaths`, `showkills`, `showonlinetime`, `showguild`, `showadmins`, `unstuck me`, `toguild:`, `toall:`.
- `makepass` (Linux ELF helper) **[bin]**: prints `crypt(3)` of its argument, so `encrpass` is traditional DES crypt. It picks the two salt characters with `rand()` but never seeds it, so every hash it makes uses the same salt, `Ng`.

Graal 1.1 Guildmaker (1999) [bin]

`Guildmaker1.1.exe` writes `guildname.code`; the later `Guildmaker.exe` adds "Save As" for any `.code` file. The Graal 1.1 client reads the same format for `guildname.code` (used by `playeringuild`) and for its registration key `reg.code`.

Format, 48 bytes:

1. Plaintext block of 24 bytes: `len` (one byte), up to 23 bytes of text, random padding.
2. Bytes 0–23: that block encrypted with key `AXBORG15`. Bytes 24–47: the same block encrypted with key `VERGRAAL`.
3. The cipher is the same LSB-first-key DES variant as the RemoteControl password (three ECB blocks).
4. On load both halves are decrypted and must match byte for byte, otherwise the file is rejected.

The sample files decode to `Baddy` (`guildname.code`), `if (playersays(killall)` (`npc.code`, cut at 23 characters), and the first bytes of a `GR-V1.03` level (`level1.graal` and `level1.graal.code`, which are identical).

`live-runs/2gta-222-2026-10-30/README.md`

Real 2.22 client on Graal the Adventure (2026-10-30)

A short live run of the real client, `Graal_222.exe` from the Graal 2.2.2.0 install, on the public server Graal the Adventure (`<public-host-2gta>:14913`), signed in as the user's own account from the client's saved login. The run was driven by a script on a personal PC (machine details redacted): about 2 minutes of walking, one door and one sword swing in the first pass, then about 6 minutes at one bush in the second. Screenshots are in this folder, captured from the window at 2.5x scale (one 16 px tile is 40 px).

Future runs: loopback-only labs (the loopback address / a second loopback address). Do not repeat personal-machine or public-listserver registrations without Squirrel's written OK.

What follows is what the screen showed. Nothing here was read from packets.

Server list



server list

- Window title "Graal Reborn Server list". The footer greets the account ("Welcome to Graal Reborn, Seji. There are 10 server(s) online.").
- Gold tab: Graal the Adventure (0 players) and GServer-RS (0 players). Silver and Bronze were not opened.
- Graal the Adventure's row: language English, version "Beta Version: 4.0.0", description "Occupied by Kalzor.", homepage `https://<graalin-site>/c/playerworlds`….
- GServer-RS's row: version "Version: 0.1.262", description "Rust gserver testing", homepage on github.com (vinvicta's gserver).

First screen after Connect



start

- The client opens a separate window titled with the server name, "Graal the Adventure". The empty strip at the bottom is the chat line.
- HUD: the A, S and D slots top left (S shows the sword with a "1", D a ring), the magic bar under them, the rupee/bomb/arrow counters in the middle, three hearts and the `00/01` counter on the right.
- Right after the level appears the counters read 0 rupees, 00 bombs, 03 arrows. A few seconds later (first screenshot after walking, walk1) they read 6, 05, 03, and later 7, 10, 03. The real client shows the counts as they arrive, after the level is already drawn; the start screen is not held back for them.
- A server-side minimap is drawn at the bottom left, with the player's head as the marker. It is a weapon/NPC image, not a built-in client panel.
- An NPC message ("Come one come all! The house of bananas is open…") scrolls across the bottom of the view in an outlined font.

Camera

Walking right or up moves the view, not the player: the player stays at the same place on screen (about the window's middle) while the board scrolls under them, until the level edge. Compare gta-start with walk1.

Level-name banner



banner

Walking up the stairs north of the start crosses into the next level. The client then shows a parchment panel, "US Sparring Complex", in the lower half of the view, over the board, for a few seconds. It is drawn in the same panel style as a sign. It is most likely a server script (the level's or a weapon's); this run did not check whether the client itself names levels on entry.

Door, water and other NPCs



inside

- Walking up into the complex's door loads the interior. The player arrives in the deep water on the left and is drawn with the swim sprite (head and shoulders only).
- NPCs carry name labels under them in the outlined font ("Marge", "Gemetzel Haus' Fighter"). The fighter shows a green "?!" above it.
- Walking down out of the interior puts the player outside on the path south of the building (down1), facing down with the shield in front.

Sword



sword

One S press, captured about 90 ms in: the sword frame facing down with the blade tip below the player. #### Second pass: bushes, hiding, chat (20:22 to 20:28 local)

Done next to the bush east of Cat's Pub. Crops are 600 x 450 physical pixels around the player.

Lift, hide under it, throw

Step	Screenshot	What the screen showed
A facing the bush	lift-b	The bush goes over the head and the 2 x 2 stump is left on the board at once. The carried bush is drawn green, while the bush on the board was the bluish-grey variety: the carried image is the plain bush sprite, not the board's tiles.
D while carrying	duck-b	The player disappears under the bush; only the bush is drawn, at ground level, on the player's spot.
Left arrow while hidden	hidemove	The bush moves with the player (one tile in 250 ms), still with nothing of the player showing.
D again	unhide	Back to the normal carry pose. D toggles.
A	throw-a	The bush is thrown in the facing direction (left) and leaves the view.

Bush respawn on this server

Lifted at 20:26:34.7 and thrown away, then the bush's spot was captured every 2.5 s for 100 s (resp-montage). The stump shows at 3, 6, 8, 11 and 13 s; the bush is back at 16 s and stays. So the server puts a lifted bush back about **14 to 16 s** after the lift. That fits the GServer-v2 rule in `client-internals/board-respawn.md` (exactly `respawntime`, default 15 s), not the 1.3-era 6 to 12 s random timer. A first, untimed lift on the same bush (20:22) had also come back within a minute.

Sword on a bush

Screenshot	What it shows
cut-a, about 100 ms after S	The sword arc to the right and four green leaf pieces flying out from the bush's spot; the stump is already on the board.
cut-b, 700 ms later	Leaves gone, stump left.

The leaf pieces are green although this bush is the bluish-grey kind.

Pushing

Holding right into the bush (face2) shows the push pose: arms forward, body leaning into the bush. Nothing moves.

Chat

- Tab opens the chat line at the bottom; the typed text shows there in the system font (chat-typing).
- Enter sends it ("hi"). The text then shows above the player's head as small white text with a dark outline, with **no bubble or box around it** (chat-a). On this server, with the 2.22 client, "chat bubble" is just outlined text.
- The line was still over the head about 45 s later, while the player had not moved. That matches Seihai's note that your own line is held until a direction key (`chat-lists.md`).
- After Enter the chat line keeps the keyboard: the arrows did not move or turn the player until Tab was pressed again. A Seihai profile for 2.22 should do the same.

Third pass: memory and packets (20:30 to 20:38 local)

Server list, all tabs

The Silver tab lists Doomsday and the Bronze tab lists "Jojo???s" (a name with non-ASCII characters, shown as `???`) and Testbed, all with 0 players (silver, bronze). The right-hand details panel does **not** change when the tab changes: it keeps showing the server picked last (Graal the Adventure). The listserver reply in the capture names 9 servers: Graal the Adventure (14913), Sharp3D TEST (14917), Testbed (14842), Login DEV (14914), Login Mobile (14912), Jojo???s (30903), Doomsday (14802), Login (14911), GServer-RS (14801). Four of them (Sharp3D TEST and the three Login servers) are on no tab, while the footer says "There are 10 server(s) online".

On the reconnect the HUD counters were right from the first frame (7 rupees, 10 bombs), unlike the very first connect of the day, where they started at 0.

The 2.22 player object in memory

Read-only, with `tools/mem.py` (ReadProcessMemory, no debugger) and `tools/findptr.py`, following the live-trace thread's recipe. Build: `Graal_222.exe` from the 2.2.2.0 install, image base 0x400000.

- A differential scan (stand, step left, stand, step right) leaves the x double at heap `0x2738008` with y right after it (x 45.0, y 39.0 on the bush spot).
- The object starts 4 bytes earlier, at `0x2738004`. Its VMT is `0x527FEC`. The static globals `0x5916B0` and `0x59A484` point at it, so **`[0x5916B0]` is the 2.22 player**, the counterpart of 2.05 rev 2's `0x54B7E4`.

A 22 s trace at 4 ms (walk right into the bush, hold right 2 s, sword, A twice, walk left) changed these fields:

Offset	What it did	Reading
+0x04 (double)	44 → 44.5 → 45 → 45.5 every 50 ms while walking, 45.5 → 42 walking back	**x**, tiles, 0.5 per 50 ms tick
+0x0C (double)	constant	**y**
+0x19C (double)	same values as x, same ticks	a second copy of x (the 2.05 object keeps x before the step at +0xFC)
+0x228 (double)	25.5 while x was 44, moving with x	the view's left edge in tiles (x − 18.5)
+0xFC	3 while facing right, 1 after turning left	**direction** (0 up, 1 left, 2 down, 3 right)
+0x044	0..7, +1 per tick while walking	**walk frame**
+0x154	0/1 alternating each step	step parity
+0x180	1 while a direction key is held, 0 after	walking flag
+0x1DC	counts 1, 2, 3 … per tick while pushing into the bush; at 19 the push frames start; it runs to 37 and drops back to 18 while the key is still held	**blocked-step counter**: the push starts after **18 blocked ticks (0.9 s)**, the same 18 as 2.05
+0x048	0..3 cycling every tick from the moment the counter passed 18	push frame
+0x2AC	byte flags: 0x100 for one tick at the sword, 0x10000 for one tick at the lift and again at the throw	action flags
+0x258	+1 every 50 ms the whole time	**tick counter** (2.05: +0x1F8)
+0x34C	16 → 0, −1 per tick, then again	a 16-tick countdown, not tied to any key
+0x160 / +0x168	812 → 813 on the walk right, back to 809 on the walk left	unknown, follows x

So 2.22 keeps 2.05's movement model: 0.5 tile per 50 ms tick, a 0.9 s push threshold, an 8-frame walk cycle and a 50 ms tick counter.

What the client sends while walking and cutting

Capture with Wireshark's `dumpcap` on the Wi-Fi adapter from before the login, decoded with `seihai capture show FILE --version 2.22`. The game connection decodes completely (gen 5, key taken from the login frame). The account and password are masked by Seihai and not kept here. Excerpt: walk-sword-c2s.txt.

Walking right 450 ms from x 42:

```
+32392ms  FLAGSET  gr.x=42         PLAYERPROPS GANI=cn_walk X=42 SPRITE=3
+32435ms  FLAGSET  gr.x=42.5
+32486ms  FLAGSET  gr.x=43         PLAYERPROPS X=43
+32539ms  FLAGSET  gr.x=43.5
 …                                  (every 50 ms)
+32741ms  FLAGSET  gr.x=45.5
```

```
+32792ms                          PLAYERPROPS X=45.5
+32892ms                          PLAYERPROPS GANI=cn_idle
```

- **A `gr.x` flag goes to the server on every 50 ms tick.** That is a Graal the Adventure script (a `gr.` client flag), not the client's own movement code. The real client sends it uncoalesced, one message per tick.
- The X prop goes on whole tiles only (every other tick) plus the final half-tile spot when the player stops. The gani switches to `cn_walk` with the first step and back to `cn_idle` 100 ms after the last.
- The server's ganis are `cn_idle`, `cn_walk`, `cn_sword`, not `idle`/`walk`/`sword`.
- The sword sends the cut at once, in the same frame as `GANI=cn_sword`: `BOARDMODIFY` with the 2 × 2 leaves (`PH""%E%F%U%V`). `cn_idle` follows 240 ms later.
- With nothing else happening the client sends one 1-byte message (id 24, byte 0x29) every 5.1 s and the server sends NEWWORLDTIME every 5.0 s. Seihai's table calls the client's id 24 `SHOWIMGPLAYER`, which does not fit a 1-byte keep-alive; the table entry is worth checking.
- At login the client sends ADJACENTLEVEL for all 8 neighbours of `westocean_m-16.nw` in one frame, then ~30 FLAGSETs and 3 FLAGDELs (the script flags it holds), then its PLAYERPROPS (`HEADGIF=head162.png`, `COLORS=11,18,18,11,11`, `OSTYPE=wind`).

For Seihai: Seihai #159 stopped Seihai from re-sending unchanged `lastweapon` flags and looks. That is still right: the real client sends a flag only when a script changes its value. The per-tick `gr.x` is the script changing it every tick, not the client repeating itself.

Not done

Pots, signs and appearance (clothes-changer) commands are still untested. A lift attempt in the third pass happened on a stump (the sword had already cut the bush), so the capture has no lift or throw message.

Fourth pass: LAN test world (21:08 to 21:16 local)

The user ran the patched GServer-v2 beta4 pack on a personal Linux host (`<lan-host>:14802`; hostname redacted) with a test level `testworld.nw`. A hosts line on that machine pointed `<list-host-reborn>` at it. Registering the same server with the public Graal Reborn listserver worked, but the real 2.22 client did not list it on any tab: that needs a ServerHQ tier. Account `seihaitest`. The traffic was captured with dumpcap from before the login. Screenshots are in lan/. **Future runs of this kind are loopback-only** (see the note at the top of this file).

Pots

Screenshot	What it shows
pot1, 120 ms after A	The pot is over the head and the pot's stand is left on the board as the empty tile. A heart is already lying in the stand.
pot2, 600 ms later	Carry pose, heart still in the stand.
pot4	Turned left and threw: the pot is gone and the heart stays where the pot was.

On this server the drop comes from the server, in the same frame as the lift: the client sends `BOARDMODIFY` and `GANI=lift`, and the server replies at once with `ITEMADD` at the pot's spot (item 5, heart). The client sends `ITEMTAKE` with the same three bytes when the player stands on it. The pot was back on the board within about 15 s (not timed precisely). (The drop is the server's `vasesdrop`/`bushitems` option; a server without it drops nothing.)

The heart-refill death loop

In the user's first session the server saved the account with 0 max hearts, 0 arrows, 0 bombs and 0 gralats, so every respawn was clipped to 0 hearts and the player died over and over. In the captured second session the 2.22 client **never sends MAXPOWER, CURPOWER, arrows, bombs or gralats**: its login PLAYERPROPS are NICKNAME, HEADGIF, COLORS, STATUS, UDPPORT, ADDITFLAGS, PLANGUAGE, OSTYPE and TEXTCODEPAGE, and later only GANI, X, Y, SPRITE, CARRYSPRITE and the clothes props below. The server's heart pickup path cannot zero max hearts (confirmed by reading the beta4 source on that lab host). So the zeroing in the first session is still unexplained; it was not captured.

Signs

- The level's `SIGN` record names the **bottom-left** tile of the 2 × 2 sign. With the record at the top-left (20,30) the sign never opened; after the test server moved it to 20,31 it works.
- Walking up into the sign from below (x 20, y 31, facing up) opens it at once, with no key press (sg1). The scroll shows **three source lines per page**, one panel line each. Lines are **not wrapped**: a long line runs past the right edge of the scroll and off the panel.
- Capital letters are drawn in green and the rest in black, as in the sign font image.
- A goes to the next page (sg2, page four is a single line). A on the last page closes the sign. While the sign is open A does not lift.
- Before the fix, A lifted the sign tile (glove 3) (sign-a), and A again threw it, breaking it into green leaf pieces (sign-b). The sign tile comes back like a bush. So on 2.22, if no readable sign covers the tile, a liftable sign tile is just a liftable object.

Walking into corners

Walking left into the bottom corner of the sign moved the player down by 0.5 tile (y 30.3 → 30.8), and walking up past the sign's corner from x 18.5 moved the player left to x 17.9. The 2.22 client slides the player around an obstacle's corner instead of stopping it.

Clothes changer

Touching the NPC changes the outfit at once (npc0 before, npc1 after) and the NPC shows "Outfit N of 4" over its head in the outlined font. The NPC's script runs on the client: each touch makes the client send one PLAYERPROPS with all of the new look together, e.g.

```
C>S PLAYERPROPS  HEADGIF=image=head2.png  COLORS=3,7,18,4,0  X=42.5  BODYIMG=body3.png
C>S NPCPROPS     (18 bytes: the NPC's own counter)
```

The server sends nothing for the change itself. So `setplayerprop`/`sethead`/`setcoatcolor`-style commands in an NPC are applied locally first and then reported as one HEADGIF+COLORS+BODYIMG message. Seihai's laundry item (appearance commands logged as "effect not applied") should do the same.

Baddies, death and respawn (21:18 to 21:21)

Six baddies were added to the test level (gray, blue, red, shooter, frog, golden warrior). Capture from before the login, decoded with Seihai.

- **The leader client runs the baddies.** The server sent `ISLEADER` at login. From then on the 2.22 client sent `BADDYPROPS` (id 15) about **60 a second** (300 per 5 s) for the six baddies. The server only relays them.

- The baddies swarmed the player at once. They speak in the outlined font ("Halt, intruder!", "You got me...") and the red and golden ones carry swords (b1). Hearts went 3 → 1.5 → 0 in a few seconds.
- **Death** (b2): the board goes black and an "End" scroll in the sign style appears under the player. The client sends `PLAYERPROPS STATUS=28` (0x08 = dead). The server answers by setting rupees, arrows and bombs to 0 and adding four items (`ITEMADD`) around the spot where the player died: the counts are dropped on the ground.
- **The client stops sending `BADDYPROPS` the moment it dies.** In 34 s of being dead it sent only the 1-byte keep-alive.
- **Respawn waits for a key.** The death screen stayed up until A was pressed (34 s here). Then the client, not the server, did the respawn, in one frame:

```
C>S LEVELWARPMOD   (back to the start spot)
C>S TAMPERCHECK
C>S FLAGSET gr.x=10, gr.y=31
C>S PLAYERPROPS CURPOWER=6 GANI=idle X=10 Y=31 SPRITE=0 STATUS=20
S>C PLAYERPROPS X=10 Y=31 X2=10 Y2=31
S>C PLAYERPROPS CURPOWER=6 STATUS=20
```

So on respawn **the 2.22 client sends CURPOWER itself**, set to full hearts (6 half-hearts for 3 hearts), and the server takes it. That is the path through which the user's first session could have saved 0 hearts: if the client respawned while it held 0 max hearts, it would send CURPOWER 0. It is a lead, not a capture of the bug. - After the respawn (d6) the player carries a name label ("Seihai-tester") under the sprite. The baddies stayed frozen where they were. The client never resumed `BADDYPROPS` and the server sent no new `ISLEADER`, so standing inside a baddy and swinging at it did nothing on either side (g4), and nothing hurt the player either.

Against Seihai's laundry list

Seihai item	What the real 2.22 client showed
First-load warps land at the link's arrival (laundry 2026-10-29-b, priority 1)	The interior was not cached and the player came in inside it, in the water. Whether that spot is the link's new-x/new-y was not checked against the level file.
Big map neighbours (priority 7)	Crossing north from the start level into the sparring complex showed no blank board or pause in the screenshot taken right after.
Sign input (priority 4)	Not tested: the one sign approached was not opened.
Duck while carrying (priority 4)	D hides the player under the carried bush, the bush moves with the player while hidden, and D again brings the player back. Seihai commit 0ddbc35 ("Hide under a carried bush or stone with D") matches this.
Bush respawn	About 15 s on this server (see above). Seihai shows the server's own restore through `PLO_BOARDMODIFY`, so nothing to change.
Chat repeat, chat bubble look	Chat over the head is plain outlined text with no bubble. Key repeat while typing was not tested.
Pots, signs, appearance commands	See the fourth pass: pots drop server items, a sign opens by walking up into it, three lines a page, unwrapped, and clothes changes are client-side then sent as one PLAYERPROPS.
HUD counters	Counters start at their defaults and update in place a few seconds after the level is drawn.

Not done

- One public chat line ("hi"), with the user's go-ahead. No PMs, no items used or dropped, no sparring.
- The client was left running and logged in at the end of the run.